PREMIUM • 2026 EDITION

Spring Boot with Redis

ZERO TO HERO INTERVIEW GUIDE

Premium handwritten-style notes



CACHE • TTL • CLUSTER • STREAMS

100	100	10	20
QUESTIONS	FOLLOW-UPS	LABS	CHAPTERS

JAVA 17+ • SPRING BOOT 3.5 • SPRING DATA REDIS 3.5

FROM YOUR FIRST CACHE TO PRODUCTION ARCHITECTURE

A PRACTICAL INTERVIEW REVISION COLLECTION

THE ZERO TO HERO STUDY METHOD

Learn Redis by seeing what happens after failure.

Short answers. Understandable examples. Clear diagrams. Every follow-up answered.

100
Original interview questions

100
Answered follow-ups

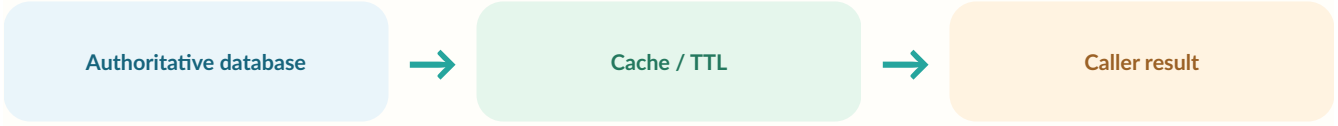
10
Spring/Redis coding labs

20
Topic chapters

HOW TO STUDY ONE QUESTION

Read the short answer → trace the Redis command → predict the crash or stale-data case → explain the follow-up without notes.

Trust boundaries matter



Java 17+ • Spring Boot 3.5 • Spring Data Redis 3.5. The source examples and questions are preserved; snippets may need imports, beans and an actual Redis server.

Practice pack: interview questions, explained diagrams, 10 source-based labs and a 30-day route. This is a learning product, not a security or correctness certification.

STUDY MAP

Your 20-chapter route

Start with fundamentals. Finish by defending production trade-offs.

01 Redis foundations and tradeoffs (Q001–Q005)

02 Data types and access patterns (Q006–Q010)

03 Keys, expiration and atomic commands (Q011–Q015)

04 Spring Boot wiring and clients (Q016–Q020)

05 Templates, serialization and repositories (Q021–Q025)

06 Spring Cache annotations (Q026–Q030)

07 Redis cache manager configuration (Q031–Q035)

08 Cache consistency and invalidation (Q036–Q040)

09 Stampedes, penetration and hot keys (Q041–Q045)

10 Transactions, pipelines and scripts (Q046–Q050)

11 Distributed locks and fencing (Q051–Q055)

12 Rate limiting and idempotency (Q056–Q060)

13 Redis Cluster (Q061–Q065)

14 Replication, Sentinel and durability (Q066–Q070)

15 Memory, eviction and performance (Q071–Q075)

16 Sessions, Pub/Sub and Streams (Q076–Q080)

17 Reactive Redis and asynchronous boundaries (Q081–Q085)

18 Testing, observability and security (Q086–Q090)

19 Production troubleshooting scenarios (Q091–Q095)

20 Senior architecture and design challenges (Q096–Q100)

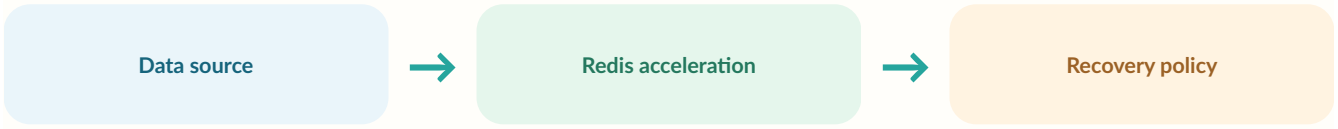
Bonus: a complete starter application, 10 source-based coding exercises, an interview checklist and a 30-day schedule.

01

CHAPTER 01 / 20

Redis foundations and tradeoffs

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q001

What is Redis and why use it with Spring Boot?

Q002

Is Redis only a cache?

Q003

Why is Redis fast?

Q004

How does Redis differ from a relational database?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

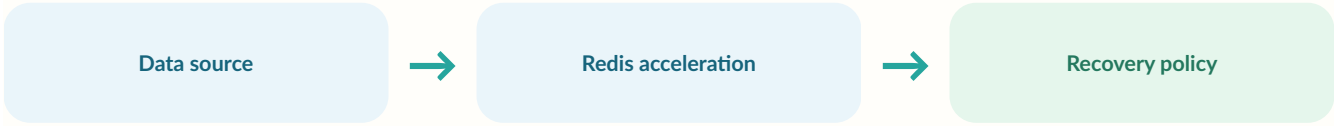
REDIS FOUNDATIONS AND TRADEOFFS

What is Redis and why use it with Spring Boot?

01 SHORT INTERVIEW ANSWER

Redis is a low-latency data store with useful data structures; Spring Boot supplies configuration and Spring Data integration.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Application -> Redis cache -> fewer repeated database reads
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Common uses include caching, sessions, counters and short-lived coordination. Each use needs a different durability and failure policy. A catalog cache can be rebuilt; losing a session or reservation can affect users directly.

05 COMMON INTERVIEW TRAP

Do not treat every Redis key as disposable cache data.

06 FOLLOW-UP: WHICH OF YOUR REDIS KEYS COULD BE SAFELY LOST?

Product cards, search suggestions and derived rankings can usually be rebuilt from the database. Sessions, reservations, payment decisions and audit records need an explicit recovery policy; never assume they are safe to discard.

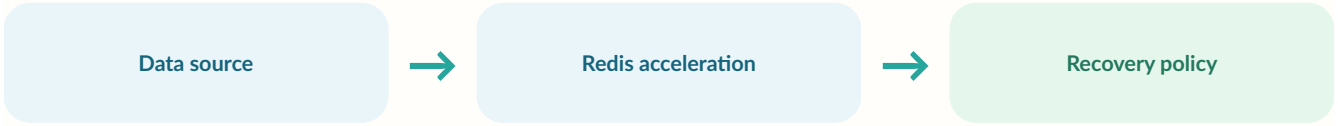
REDIS FOUNDATIONS AND TRADEOFFS

Is Redis only a cache?

01 SHORT INTERVIEW ANSWER

No. Redis can hold primary application state, but correctness then depends on persistence, replication, backup and recovery choices.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Rebuildable product cache != sole payment ledger
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Separate state by business importance. If Redis is authoritative, define acceptable loss, restore tests and write semantics. A relational database can remain the durable source for business transactions while Redis accelerates reads.

05 COMMON INTERVIEW TRAP

Enabling persistence is not a complete recovery strategy.

06 FOLLOW-UP: HOW WOULD YOU CHOOSE THE AUTHORITATIVE STORE FOR BALANCES?

Keep monetary balances and transfers in a durable store with transaction isolation, constraints, auditability and tested recovery. Use Redis for fast derived reads only unless its durability model demonstrably meets the financial requirement.

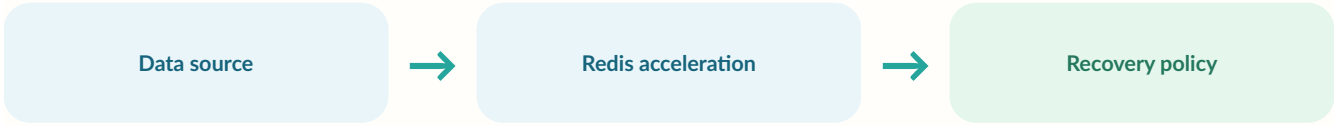
REDIS FOUNDATIONS AND TRADEOFFS

Why is Redis fast?

01 SHORT INTERVIEW ANSWER

Many operations work on in-memory data with efficient structures and low protocol overhead.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
GET product:42 -> one keyed lookup, no table scan
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Network round trips, large payloads and command complexity still matter. Long commands or scripts can delay unrelated work. Redis uses multiple threads for some activities, so describing the entire server as universally single-threaded is inaccurate.

05 COMMON INTERVIEW TRAP

In-memory does not mean every command has constant cost.

06 FOLLOW-UP: WHAT CAN MAKE A SMALL REDIS DEPLOYMENT SLOW?

Network round trips, CPU-heavy Lua, large keys, eviction pressure, swapping, persistence I/O, slow clients and hot-key contention can all increase latency. Check server latency and client/network timings separately.

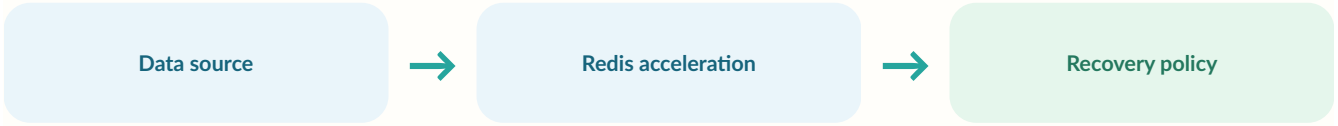
REDIS FOUNDATIONS AND TRADEOFFS

How does Redis differ from a relational database?

01 SHORT INTERVIEW ANSWER

Redis favors key-oriented structures and direct operations; relational databases offer rich relational queries and transaction/invariant tools.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Redis: GET catalog:v1:42
   SQL: SELECT ... JOIN ... WHERE ...
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Model Redis access paths deliberately instead of reproducing every relational join with round trips. Use database constraints for durable business rules when appropriate. The choice depends on query patterns, correctness and operational needs.

05 COMMON INTERVIEW TRAP

Redis MULTI is not equivalent to all SQL ACID behavior.

06 FOLLOW-UP: WHICH INVARIANT WOULD YOU KEEP IN SQL?

Keep rules such as unique payment reference, nonnegative inventory or balanced ledger postings in a transactional database. Cache a projection in Redis; do not make cache expiry the only enforcement mechanism.

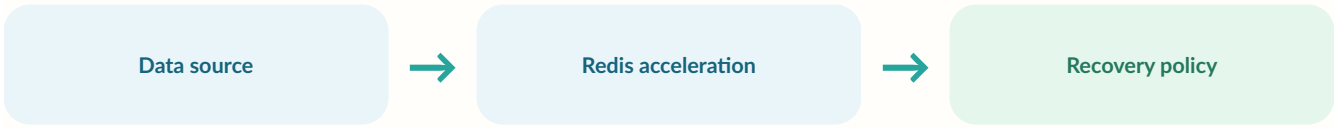
REDIS FOUNDATIONS AND TRADEOFFS

What is the difference between local and distributed caching?

01 SHORT INTERVIEW ANSWER

A local cache lives in one process; Redis can serve many application instances with shared state.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Pod A Caffeine entry != Pod B Caffeine entry
   Both pods can read the same Redis key
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Local hits avoid network latency but need separate invalidation and memory management. Shared Redis reduces duplicate storage while adding network and service availability dependencies. A two-level cache inherits both sets of tradeoffs.

05 COMMON INTERVIEW TRAP

Shared storage alone does not make cached values always fresh.

06 FOLLOW-UP: WHEN WOULD YOU COMBINE CAFFEINE AND REDIS?

Use Caffeine for extremely frequent, short-lived in-process reads and Redis for a shared cross-pod cache. Define L1 invalidation or very short TTLs, since local entries can remain stale after Redis changes.

02

CHAPTER 02 / 20

Data types and access patterns

Five focused questions, with concrete Redis behavior and safe failure boundaries.

Access need → Redis data type → Cost and limits

What you will be able to explain

Q006
When should you use a string value?

Q007
When should you use a hash?

Q008
When should you use lists and sets?

Q009
What is a sorted set useful for?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

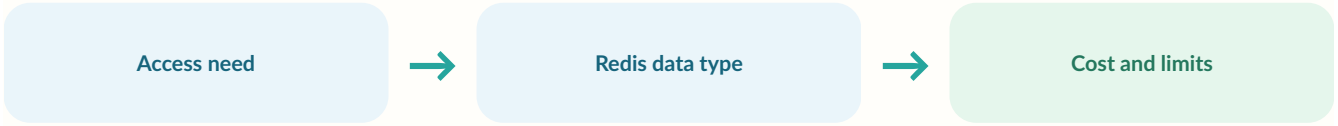
DATA TYPES AND ACCESS PATTERNS

When should you use a string value?

01 SHORT INTERVIEW ANSWER

Use strings for simple values, serialized objects and operations such as counters.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SET product:42 '{"name":"Pen"}' EX 60
   GET product:42
   INCR page:view:42
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Redis strings are byte sequences, not automatically JSON documents. A JSON value usually needs whole-value serialization unless a separately supported JSON feature is used. Use compact DTOs and explicit schemas.

05 COMMON INTERVIEW TRAP

GET does not query fields inside ordinary JSON strings.

06 FOLLOW-UP: WHEN WOULD A HASH BE BETTER?

A hash fits when callers frequently update or fetch individual fields, such as HSET profile:42 city Chennai. Use a string for a single serialized value that is usually replaced as a whole.

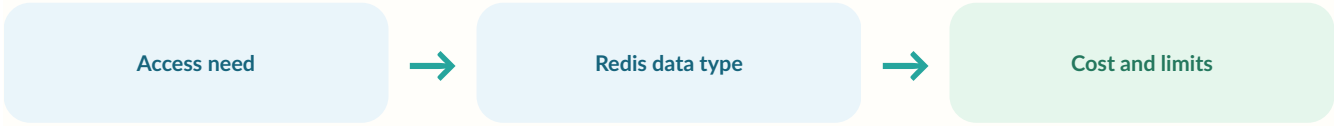
DATA TYPES AND ACCESS PATTERNS

When should you use a hash?

01 SHORT INTERVIEW ANSWER

Use a hash for a map of fields that can be read or updated separately.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
HSET profile:42 name Sai city Chennai
   HGET profile:42 city
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Field operations reduce whole-object rewriting. Key expiry affects the hash key; newer Redis versions may offer field-expiration features, but support must be checked against server/client versions. A partial read requires clear missing-field semantics.

05 COMMON INTERVIEW TRAP

Do not assume every Redis deployment supports hash-field TTL.

06 FOLLOW-UP: HOW WOULD YOU EVOLVE FIELDS SAFELY?

Add optional fields with sensible reader defaults, retain old names during migration, and version or dual-read incompatible formats. Verify old and new applications against existing keys before rolling out.

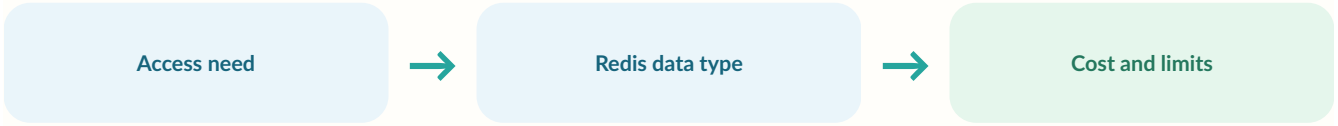
DATA TYPES AND ACCESS PATTERNS

When should you use lists and sets?

01 SHORT INTERVIEW ANSWER

Lists preserve element order and allow duplicates; sets represent unique membership without insertion-order guarantees.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
LPUSH recent:42 order-1 order-2
   SADD user:42:roles viewer editor
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Lists fit bounded recent history or queue-like patterns. Sets fit membership and set algebra. Neither implies a durable acknowledgment workflow for work processing. Bound growth and evaluate command complexity for large collections.

05 COMMON INTERVIEW TRAP

A plain list pop can lose work if the worker dies.

06 FOLLOW-UP: WHAT WOULD YOU USE FOR RECOVERABLE WORK DELIVERY?

Use Redis Streams with consumer groups and pending-entry recovery, or a durable messaging service such as Kafka. A plain list pop or Pub/Sub notification lacks the same acknowledgment and replay semantics.

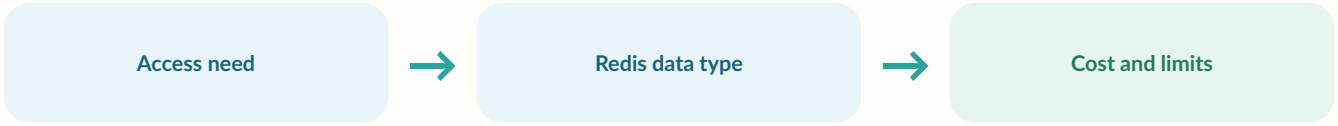
DATA TYPES AND ACCESS PATTERNS

What is a sorted set useful for?

01 SHORT INTERVIEW ANSWER

A sorted set stores unique members ordered by score, useful for ranking and time-based selection.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
ZADD leaderboard 120 user-42 95 user-7
   ZREVRANGE leaderboard 0 9 WITHSCORES
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Scores are numeric, with tie ordering determined by Redis rules. Leaderboards need explicit tie semantics and score-update policies. Avoid treating floating-point scores as a universally exact monetary representation.

05 COMMON INTERVIEW TRAP

Sorted-set rank is not an authorization decision.

06 FOLLOW-UP: HOW WOULD YOU REMOVE OLD ENTRIES FROM A SLIDING WINDOW?

Use ZREMRANGEBYSCORE to remove scores older than the window, then count or rank the remaining entries. Perform cleanup and admission atomically in a bounded Lua script when the outcome must be consistent.

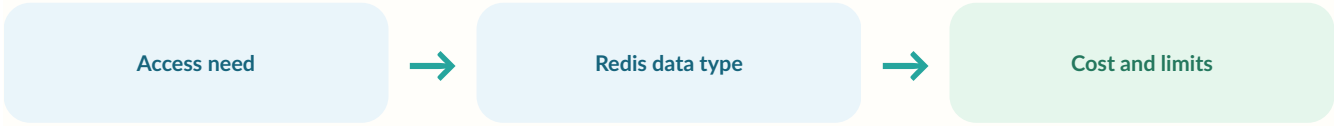
DATA TYPES AND ACCESS PATTERNS

When do bitmaps and HyperLogLog help?

01 SHORT INTERVIEW ANSWER

Bitmaps represent compact positional flags; HyperLogLog estimates distinct cardinality with bounded memory.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SETBIT active:2026-10-08 42 1
   PFADD visitors user-42 user-7
   PFCOUNT visitors
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Bitmaps need stable numeric positions and can become large if offsets are sparse. HyperLogLog is approximate and cannot enumerate individual members. Choose them when the operation fits these limitations.

05 COMMON INTERVIEW TRAP

An approximate visitor count cannot replace exact billing counts.

06 FOLLOW-UP: WHAT DATA WOULD REQUIRE A NORMAL SET INSTEAD?

Use a regular Set when you must know whether a particular user exists, enumerate exact members, or calculate an exact small membership count. HyperLogLog cannot list members or provide exact cardinality.

03

CHAPTER 03 / 20

Keys, expiration and atomic commands

Five focused questions, with concrete Redis behavior and safe failure boundaries.

Key identity → Atomic command → Expiry behavior

What you will be able to explain

Q011
How should Redis keys be designed?

Q012
How do TTL and PTTL behave?

Q013
Why set value and expiry in one command?

Q014
What happens to TTL when a value is overwritten?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

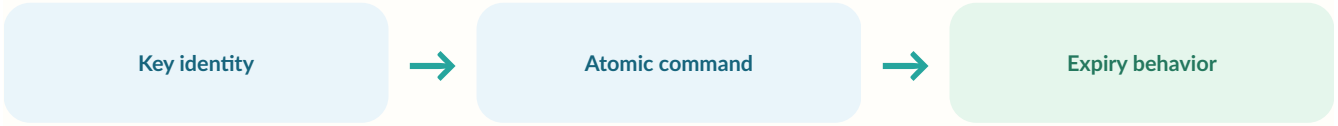
KEYS, EXPIRATION AND ATOMIC COMMANDS

How should Redis keys be designed?

01 SHORT INTERVIEW ANSWER

Use a stable namespace that includes the application, purpose and required identity boundaries.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
shop:prod:product:v1:tenant-7:42
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Include tenant and representation versions when needed. Keep sensitive identifiers out of exposed key names where possible. A consistent namespace supports inspection and targeted cleanup, but is not access control by itself.

05 COMMON INTERVIEW TRAP

Two services using the same unqualified key can overwrite each other.

06 FOLLOW-UP: HOW DO YOU MIGRATE A KEY FORMAT?

Introduce a v2 namespace, deploy readers that can read v2 and v1, backfill or lazily populate v2, then switch writes. Monitor misses and preserve old keys until rollback is no longer needed.

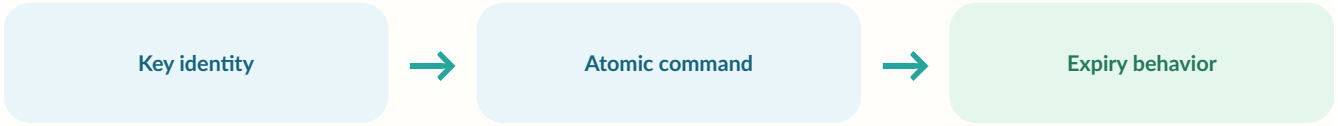
KEYS, EXPIRATION AND ATOMIC COMMANDS

How do TTL and PTTL behave?

01 SHORT INTERVIEW ANSWER

They report remaining lifetime in seconds or milliseconds; special results identify missing keys and keys without expiration.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
TTL key -> -2: key absent
   TTL key -> -1: key exists without expiry
   PTTL key -> positive milliseconds remaining
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Expiration is a logical access boundary managed through passive and active expiry. A key may expire between two commands, so do not assume a positive TTL from an earlier read proves later existence.

05 COMMON INTERVIEW TRAP

Redis TTL is not a precise business scheduler.

06 FOLLOW-UP: HOW WOULD YOU ENFORCE A RESERVATION DEADLINE DURABLY?

Store reservation expiry and status in the authoritative database and use a scheduled or event-driven expiry worker with idempotent transitions. A Redis TTL can assist cleanup but is not proof that the reservation was durably canceled.

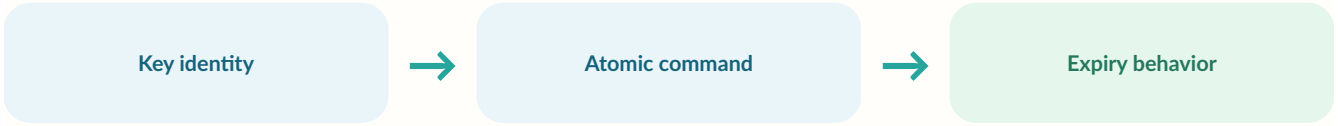
KEYS, EXPIRATION AND ATOMIC COMMANDS

Why set value and expiry in one command?

01 SHORT INTERVIEW ANSWER

A combined command avoids the crash gap between creating a value and separately setting its TTL.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SET otp:42 123456 EX 120
   // Instead of SET followed later by EXPIRE.
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A process dying between separate commands can leave an immortal key. StringRedisTemplate offers set with Duration. Other structures may need a script or a reviewed transaction to combine creation and expiry.

05 COMMON INTERVIEW TRAP

Pipelining separate commands is not atomic.

06 FOLLOW-UP: HOW WOULD YOU CREATE AN EXPIRING COUNTER SAFELY?

Use a Lua script that performs INCR and sets PEXPIRE only when the count is first created, or another single atomic protocol. Test concurrent first increments and retries after unknown outcomes.

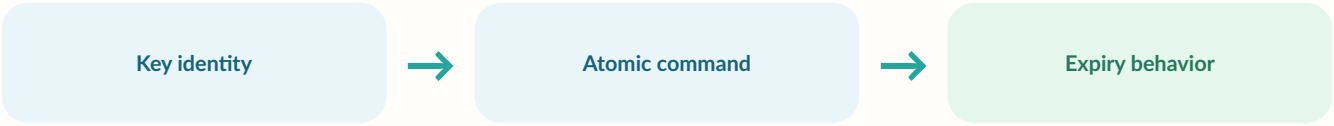
KEYS, EXPIRATION AND ATOMIC COMMANDS

What happens to TTL when a value is overwritten?

01 SHORT INTERVIEW ANSWER

A normal SET replaces the value and clears its previous expiry unless an expiry or supported preservation option is specified.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SET k one EX 60
   SET k two
   TTL k -> -1
```

04 HOW IT WORKS, IN PLAIN ENGLISH

In-place operations such as INCR or HSET ordinarily preserve the key expiry, unlike SET replacement. Verify the precise command semantics. KEEPTTL is a server-version-dependent option for retaining an existing deadline.

05 COMMON INTERVIEW TRAP

A refresh write can accidentally create a permanent cache key.

06 FOLLOW-UP: WHICH TEMPLATE OPERATION ARE YOU USING TO UPDATE VALUES?

Inspect the actual Spring Redis API: opsForValue().set(key,value) normally removes an existing TTL, while set(key,value,Duration) specifies a new deadline. For TTL preservation, use explicitly supported Redis options and test the wire behavior.

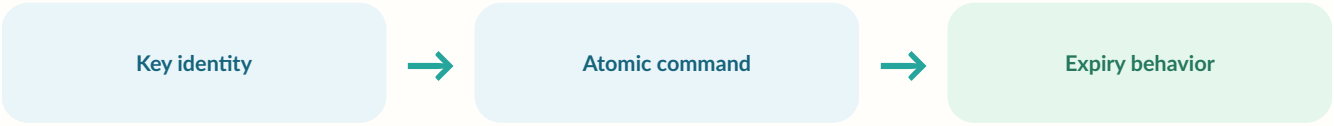
KEYS, EXPIRATION AND ATOMIC COMMANDS

Why is INCR safer than GET plus SET for a counter?

01 SHORT INTERVIEW ANSWER

INCR performs one atomic counter update, while a read-modify-write sequence can lose concurrent increments.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Two callers: GET 10; both SET 11 -> one increment lost
   Two INCR calls: 10 -> 11 -> 12
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Atomic increment does not atomically establish a TTL unless the complete policy is combined. Redis expects a supported integer representation and can reject wrong types or overflow. Define timeout/retry behavior for increments.

05 COMMON INTERVIEW TRAP

Retrying after an unknown outcome can increment twice.

06 FOLLOW-UP: WHEN WOULD YOU NEED AN OPERATION IDENTITY?

An operation ID is essential when a request may be retried after an uncertain INCR response, especially for billing or quotas. Store/deduplicate request identity and its result under an atomic, sufficiently durable policy.

04

CHAPTER 04 / 20

Spring Boot wiring and clients

Five focused questions, with concrete Redis behavior and safe failure boundaries.

Boot configuration → Redis client → Failure budget

What you will be able to explain

Q016
Which dependency enables Redis access in Spring Boot?

Q017
What properties configure a standalone Redis connection?

Q018
How do Lettuce and Jedis differ?

Q019
What does RedisConnectionFactory do?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

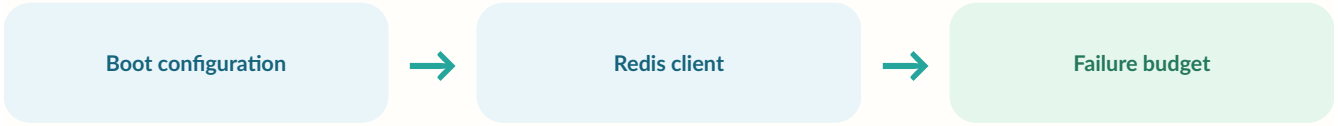
SPRING BOOT WIRING AND CLIENTS

Which dependency enables Redis access in Spring Boot?

01 SHORT INTERVIEW ANSWER

Use spring-boot-starter-data-redis for imperative access and the reactive starter for reactive integration.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
<artifactId>spring-boot-starter-data-redis</artifactId>
   // Versions managed by the Boot parent/BOM.
```

04 HOW IT WORKS, IN PLAIN ENGLISH

The starter integrates Spring Data Redis and defaults to Lettuce in the baseline. Redis server is deployed separately. Boot 3.5 and Spring Data Redis 3.5 APIs are the teaching baseline; inspect resolved versions before mixing examples.

05 COMMON INTERVIEW TRAP

Adding a starter does not start a production Redis service.

06 FOLLOW-UP: HOW DO YOU INSPECT THE MANAGED DRIVER VERSION?

Run mvn dependency:tree or mvn help:effective-pom and inspect the Boot BOM-managed Lettuce/Jedis versions. Check the selected runtime classpath, not only the dependency typed into pom.xml.

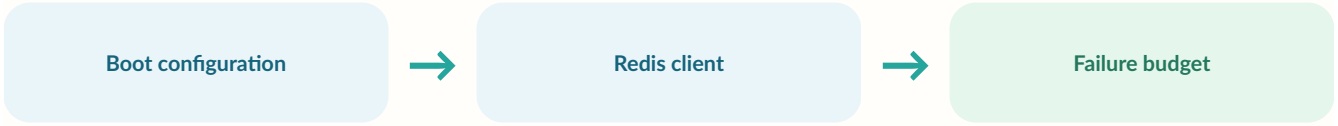
SPRING BOOT WIRING AND CLIENTS

What properties configure a standalone Redis connection?

01 SHORT INTERVIEW ANSWER

In Boot 3.5, use the spring.data.redis property namespace for host, port, credentials and timeout settings.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
spring.data.redis.host=localhost
   spring.data.redis.port=6379
   spring.data.redis.connect-timeout=2s
   spring.data.redis.timeout=1s
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Connection establishment and command wait budgets are different. Supply authenticated TLS configuration for the intended environment and avoid storing credentials in source. Old property namespaces can be misleading after migration.

05 COMMON INTERVIEW TRAP

A longer timeout can turn an outage into thread exhaustion.

06 FOLLOW-UP: WHAT BUDGET SHOULD AN API ALLOCATE TO A CACHE LOOKUP?

Reserve a small, measured fraction of the end-to-end request deadline for Redis, including connection and command time. For a noncritical cache, fail fast to a bounded fallback rather than consuming the entire request budget.

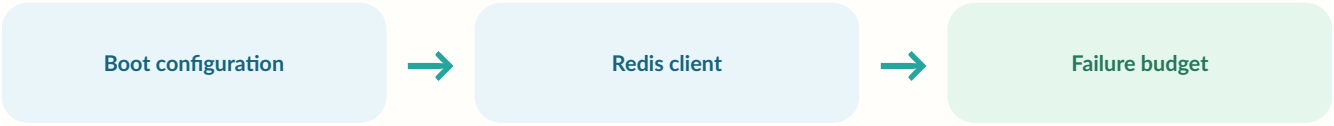
SPRING BOOT WIRING AND CLIENTS

How do Lettuce and Jedis differ?

01 SHORT INTERVIEW ANSWER

Lettuce supports asynchronous/reactive operations with a Netty-based model; Jedis has a different client and connection usage model.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Default Boot starter -> Lettuce
   Driver selection -> connection and pooling review
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Choose by required features, topology and operational behavior, not an outdated universal performance claim. Drivers evolve, so verify supported versions. Spring Data's abstraction does not erase all driver-specific cluster or scan behavior.

05 COMMON INTERVIEW TRAP

Client choice does not remove the need for deadlines.

06 FOLLOW-UP: WHICH FEATURE WOULD DETERMINE YOUR DRIVER CHOICE?

Check whether the workload needs true reactive operations, blocking commands, pipelining, Cluster/Sentinel, TLS, or pooling. Choose a supported driver version after a behavior test, not a universal speed claim.

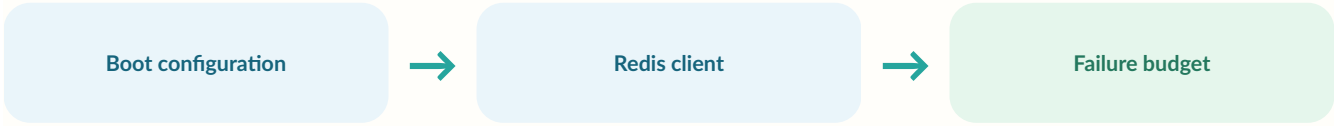
SPRING BOOT WIRING AND CLIENTS

What does RedisConnectionFactory do?

01 SHORT INTERVIEW ANSWER

It supplies Spring Redis connections and translates driver integration into Spring Data's connection abstraction.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
RedisConnectionFactory -> RedisTemplate -> opsForValue()
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Boot can auto-configure the factory from properties. Custom factories can replace defaults and require explicit lifecycle/configuration. Individual Spring RedisConnection instances should not be treated as generally thread-safe application singletons.

05 COMMON INTERVIEW TRAP

Reusing RedisTemplate is different from sharing a raw connection.

06 FOLLOW-UP: WHO CLOSES A MANUALLY OBTAINED CONNECTION?

A caller that obtains a native connection directly owns releasing/closing it according to the factory API, usually with try/finally. Spring-managed templates handle their own connection lifecycle; do not cache a raw connection as a singleton.

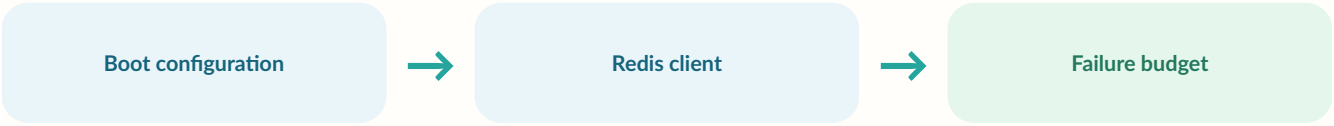
SPRING BOOT WIRING AND CLIENTS

Does Lettuce always need a connection pool?

01 SHORT INTERVIEW ANSWER

No. Ordinary operations can share a native connection; blocking and transactional use cases need separate consideration.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Simple GET/SET -> shared native connection by default
   Blocking/transactional usage -> reviewed dedicated connection handling
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Pooling support needs the appropriate pool dependency/configuration. Multiplexing reduces connection counts but does not remove head-of-line or server capacity limits. Inspect actual connection use before increasing pool size.

05 COMMON INTERVIEW TRAP

A pool of 100 does not create 100 Redis command execution cores.

06 FOLLOW-UP: WHEN WOULD BLOCKING OPERATIONS REQUIRE ISOLATION?

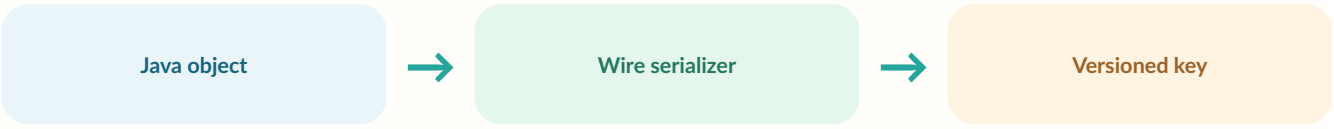
Blocking reads, long-lived Pub/Sub subscriptions, or transactions can monopolize a connection and may need isolated dedicated connections. Ordinary nonblocking GET/SET through Lettuce can often share a connection safely.

05

CHAPTER 05 / 20

Templates, serialization and repositories

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q021

How do RedisTemplate and StringRedisTemplate differ?

Q022

Why can Redis keys look unreadable in redis-cli?

Q023

How should cached JSON objects evolve?

Q024

How do hash serializers differ from value serializers?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

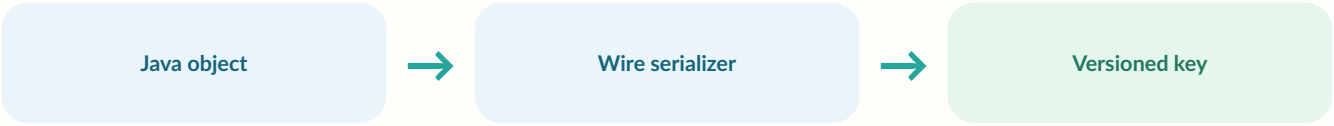
TEMPLATES, SERIALIZATION AND REPOSITORIES

How do RedisTemplate and StringRedisTemplate differ?

01 SHORT INTERVIEW ANSWER

RedisTemplate is configurable for typed key/value serialization; StringRedisTemplate uses string serialization for its normal operations.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
strings.opsForValue().set("greeting", "hello", Duration.ofMinutes(1));
```

04 HOW IT WORKS, IN PLAIN ENGLISH

StringRedisTemplate makes CLI inspection and simple JSON strings straightforward. A typed RedisTemplate needs matching key, value and hash serializers. Java generic parameters do not automatically select the wire format.

05 COMMON INTERVIEW TRAP

RedisTemplate<String,Product> alone does not configure JSON.

06 FOLLOW-UP: HOW WOULD YOU INSPECT THE ACTUAL STORED BYTES?

Read a known key with redis-cli --raw or inspect its hexadecimal bytes; compare with the configured key/value serializers. Check hash field serializers separately if using hashes.

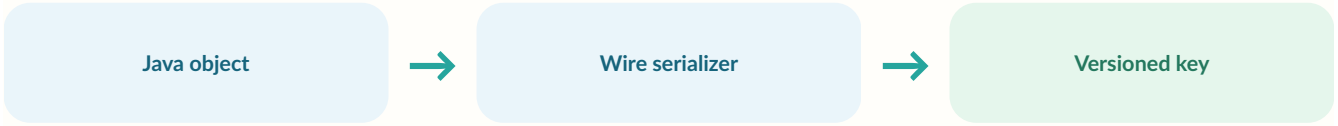
TEMPLATES, SERIALIZATION AND REPOSITORIES

Why can Redis keys look unreadable in redis-cli?

01 SHORT INTERVIEW ANSWER

The template may use binary Java serialization rather than plain string serialization.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
CLI key: binary prefix + encoded text
   StringRedisSerializer -> human-readable key bytes
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A default RedisTemplate and the Redis cache manager can have different serialization settings. Inspect each actual path. Changing serializers affects compatibility with existing data and requires migration or a versioned namespace.

05 COMMON INTERVIEW TRAP

Switching a serializer does not rewrite old keys.

06 FOLLOW-UP: HOW DO YOU MIGRATE WITHOUT BREAKING ROLLBACK?

Use a versioned namespace and dual-compatible readers; write new values without rewriting all old keys in place. Keep old-format data or a backfill/rollback strategy until the old release can be safely retired.

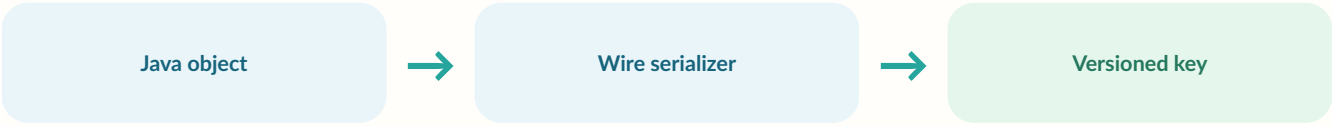
TEMPLATES, SERIALIZATION AND REPOSITORIES

How should cached JSON objects evolve?

01 SHORT INTERVIEW ANSWER

Use an explicit DTO schema, tolerant reading where appropriate and a versioned namespace for incompatible changes.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
product:v1:42 -> old DTO
   product:v2:42 -> new DTO
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Avoid caching JPA entities with lazy proxies or every internal field. Class metadata can couple values to Java implementation names; restrict accepted types. Test old stored values against the new application during a rolling upgrade.

05 COMMON INTERVIEW TRAP

Deserializing successfully does not ensure semantic compatibility.

06 FOLLOW-UP: WHICH CHANGE REQUIRES A NEW CACHE NAMESPACE?

Change namespaces when a schema breaks existing readers, changes meaning, or changes the serialized type/codec. For compatible optional fields, tolerant readers may allow the same namespace after tested rollouts.

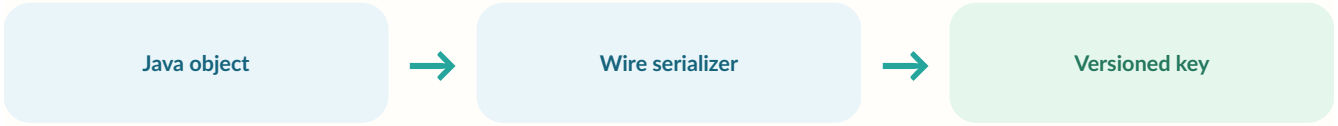
TEMPLATES, SERIALIZATION AND REPOSITORIES

How do hash serializers differ from value serializers?

01 SHORT INTERVIEW ANSWER

A hash uses separate serializers for its field names and field values, in addition to the key serializer.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
keySerializer; valueSerializer
   hashKeySerializer; hashValueSerializer
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Correct string values do not prove hash fields are encoded correctly. Configure the full template consistently and validate CLI reads. Hash mapping is a separate application concern when storing object fields.

05 COMMON INTERVIEW TRAP

Updating only valueSerializer does not fix a broken hash encoding.

06 FOLLOW-UP: WHAT BYTES WOULD HGET EXPECT FOR THE FIELD NAME?

HGET sends the serializer-produced bytes for the hash field name, which must match the bytes used by HSET. A UTF-8 string field serializer expects readable field-name bytes; Java-serialized names will not match them.

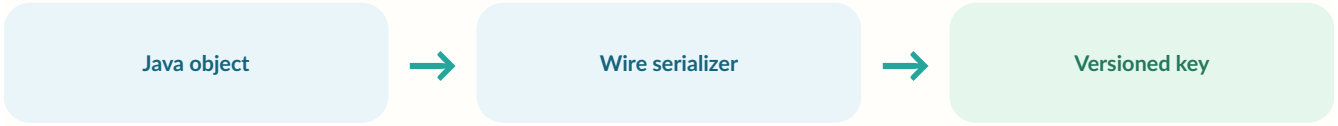
TEMPLATES, SERIALIZATION AND REPOSITORIES

What do Redis repositories provide?

01 SHORT INTERVIEW ANSWER

They map supported domain objects to Redis structures and maintain repository-specific indexes and metadata.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
@RedisHash("people")
   class Person { @Id String id; String name; }
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Repositories are a separate model from cache annotations. Indexes, TTL and keyspace-event behavior require review. They do not provide relational joins or general Redis transaction support. Select them only when their object lifecycle fits the use case.

05 COMMON INTERVIEW TRAP

A repository entity is not automatically a cached SQL entity.

06 FOLLOW-UP: HOW ARE SECONDARY INDEXES CLEANED AFTER EXPIRATION?

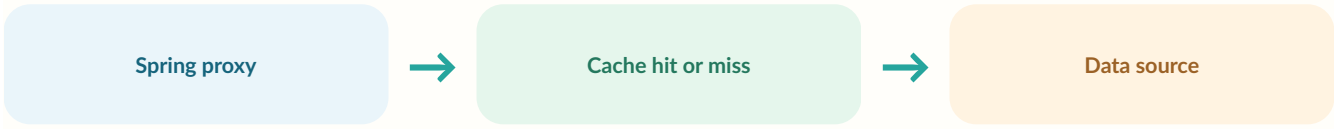
Redis repository index cleanup relies on its keyspace-event and shadow-copy behavior, and expiry events are not perfectly durable. For correctness, test cleanup and use reconciliation or explicit index maintenance rather than trusting notifications alone.

06

CHAPTER 06 / 20

Spring Cache annotations

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q026
What does @Cacheable do?

Q027
How is the default Spring cache key chosen?

Q028
How do @CachePut and @CacheEvict differ?

Q029
How do condition and unless differ?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

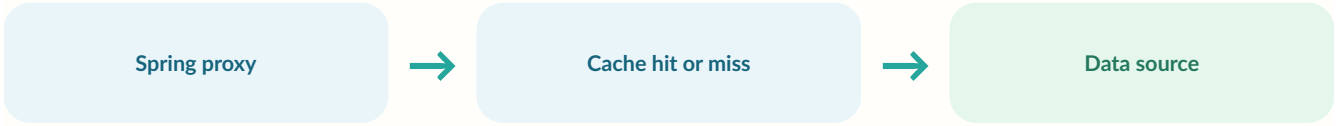
SPRING CACHE ANNOTATIONS

What does @Cacheable do?

01 SHORT INTERVIEW ANSWER

It returns a cached value for a hit and invokes the method on a miss under the selected cache provider's behavior.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
@Cacheable(cacheNames="products", key="#id")
   public Product find(String id) { return database.load(id); }
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Enable caching and select a Redis cache manager deliberately. The annotation expresses cache behavior, not a data consistency contract. Default concurrency can allow several callers to load the same missed key.

05 COMMON INTERVIEW TRAP

@Cacheable is not a distributed mutex.

06 FOLLOW-UP: WHAT HAPPENS WHEN TWO PODS MISS AT THE SAME TIME?

Both pods may see a miss and load the database at once. Ordinary @Cacheable does not provide distributed single-flight: use bounded loader concurrency, accept some duplication or choose a provider-specific coordination protocol.

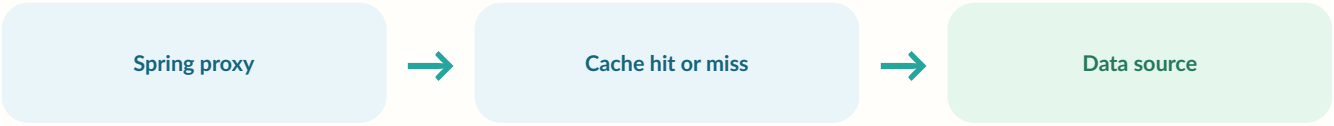
SPRING CACHE ANNOTATIONS

How is the default Spring cache key chosen?

01 SHORT INTERVIEW ANSWER

No arguments use an empty key; one argument generally uses that argument; multiple arguments use a composite SimpleKey.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
find("42") -> key based on "42"
   find("tenant-7", "42") -> composite identity
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Make important identity dimensions explicit, especially tenant, locale and representation. A custom key or KeyGenerator can standardize identity. Redis serialization/prefixing then produces the stored key bytes.

05 COMMON INTERVIEW TRAP

Omitting tenant can leak another tenant's cached result.

06 FOLLOW-UP: WHICH ARGUMENTS TRULY INFLUENCE THE RETURNED VALUE?

Include every dimension that affects the response: tenant, product ID, locale, price tier, representation version and permission scope when relevant. Never key a private response only by a public entity ID.

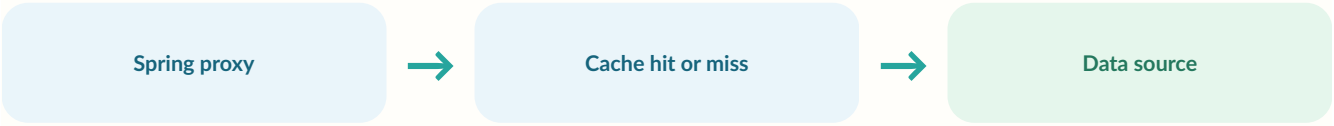
SPRING CACHE ANNOTATIONS

How do @CachePut and @CacheEvict differ?

01 SHORT INTERVIEW ANSWER

CachePut invokes the method and stores its result; CacheEvict removes selected cached data.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
@CachePut(cacheNames="products", key="#result.id")
   public Product update(...) { ... }

   @CacheEvict(cacheNames="products", key="#id")
   public void delete(String id) { ... }
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Eviction normally occurs after successful method completion unless beforeInvocation is configured. Method completion is not necessarily outer transaction commit. Review advice ordering and transaction-aware cache behavior for writes.

05 COMMON INTERVIEW TRAP

A DB rollback can leave an incorrectly updated cache.

06 FOLLOW-UP: HOW DO YOU ALIGN INVALIDATION WITH COMMIT?

Evict after successful DB commit via transaction-aware cache management or a transaction synchronization/outbox event. Then test rollback and a late reader, because post-commit eviction still allows a stale-load race.

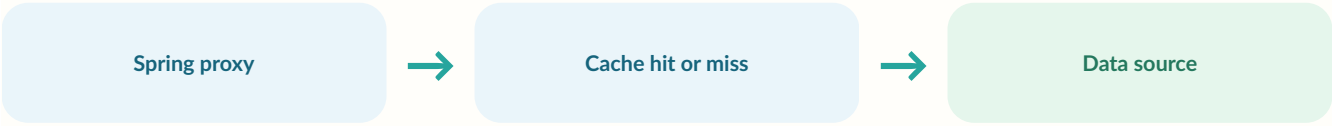
SPRING CACHE ANNOTATIONS

How do condition and unless differ?

01 SHORT INTERVIEW ANSWER

Condition decides whether caching applies based on allowed context; unless can veto storing the returned result after invocation.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
@Cacheable(cacheNames="products", key="#id",
       condition="#id != null", unless="#result == null")
```

04 HOW IT WORKS, IN PLAIN ENGLISH

The result is available after invocation for supported expressions such as unless. A cache hit does not invoke the method to recompute a result. Null policy and provider configuration should match the annotation.

05 COMMON INTERVIEW TRAP

Not every annotation option combines with sync=true.

06 FOLLOW-UP: WHEN WOULD SHORT-LIVED NEGATIVE CACHING BE PREFERABLE?

Cache a brief 'not found' result when repeated searches for nonexistent IDs overload the database and the cost of short staleness is acceptable. Invalidate promptly when a record is created.

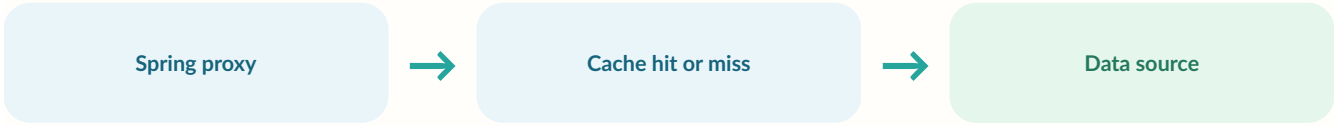
SPRING CACHE ANNOTATIONS

Why does a self-call bypass @Cacheable?

01 SHORT INTERVIEW ANSWER

Default Spring caching uses proxies, and an internal this.method() call bypasses the proxy advice.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Outer bean -> cached service proxy -> interception
   this.find(id) -> direct method call -> no default cache advice
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Put the cached operation on a separate injected service when needed. Ensure the caller receives the Spring-managed proxy and the method is interceptable. The same general boundary applies to default transaction advice.

05 COMMON INTERVIEW TRAP

Calling new ProductService() also bypasses Spring caching.

06 FOLLOW-UP: HOW WOULD AN INTEGRATION TEST REVEAL PROXY BYPASS?

Inject the cached service into another Spring bean and call it twice; the source should run once. Contrast with this.find(...) inside the bean and assert that the source runs again, showing bypass of the proxy.

07

CHAPTER 07 / 20

Redis cache manager configuration

Five focused questions, with concrete Redis behavior and safe failure boundaries.

Cache name → TTL + codec → Reader behavior

What you will be able to explain

Q031
What does RedisCacheManager manage?

Q032
Can different caches have different TTLs?

Q033
How do TTL and time-to-idle differ?

Q034
What does @Cacheable(sync=true) guarantee?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

REDIS CACHE MANAGER CONFIGURATION

What does RedisCacheManager manage?

01 SHORT INTERVIEW ANSWER

It supplies named Spring Cache instances backed by Redis with shared or per-cache configuration.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
RedisCacheConfiguration.defaultCacheConfig()
       .entryTtl(Duration.ofMinutes(5))
       .disableCachingNullValues();
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Configure namespaces, serializers and TTL by data behavior. Default Redis cache settings include no expiry unless changed and can use Java value serialization. A template's serializer settings do not automatically alter this cache manager.

05 COMMON INTERVIEW TRAP

Configuring StringRedisTemplate does not configure @Cacheable values.

06 FOLLOW-UP: WHICH BEAN CONTROLS ANNOTATED CACHE SERIALIZATION?

The RedisCacheManager bean and each RedisCacheConfiguration control annotated cache value codecs and TTLs. A separately configured StringRedisTemplate does not automatically change @Cacheable serialization.

REDIS CACHE MANAGER CONFIGURATION

Can different caches have different TTLs?

01 SHORT INTERVIEW ANSWER

Yes. Configure named caches separately instead of treating every data class as equally fresh.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
products: 5 minutes
   exchange-rates: 30 seconds
   feature-config: reviewed policy
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Per-entry TTL computation is also available through version-specific cache writer APIs, introduced in Spring Data Redis 3.2. Match TTL to tolerated staleness and load, with jitter where the implementation permits it.

05 COMMON INTERVIEW TRAP

A five-minute TTL is not a promise of five-minute fresh data.

06 FOLLOW-UP: HOW CAN A LATE STALE WRITE EXTEND STALENESS?

A reader may load old DB content before a write, then SET it after the writer commits and evicts. The late SET resets the key's TTL, extending the time during which an old value remains visible.

REDIS CACHE MANAGER CONFIGURATION

How do TTL and time-to-idle differ?

01 SHORT INTERVIEW ANSWER

TTL ends a fixed lifetime; time-to-idle extends a lifetime on qualifying access.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SET k value EX 60 -> ordinary GET does not refresh deadline
   GETEX k EX 60 -> read and set a new deadline
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Spring Redis cache can emulate TTI using GETEX on cache reads, requiring Redis 6.2+ support. All relevant access paths must refresh consistently; a direct GET does not follow that policy. TTI can retain popular stale data indefinitely without invalidation.

05 COMMON INTERVIEW TRAP

Turning on TTI does not make every Redis client read refresh expiry.

06 FOLLOW-UP: WHAT PREVENTS A HOT STALE VALUE LIVING FOREVER?

Use explicit invalidation/version checks or an absolute maximum age that reads cannot extend. GETEX-based TTI alone can keep a frequently accessed stale entry alive indefinitely.

REDIS CACHE MANAGER CONFIGURATION

What does @Cacheable(sync=true) guarantee?

01 SHORT INTERVIEW ANSWER

It requests provider-supported synchronized loading, not a universal cross-pod single-flight guarantee.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
@Cacheable(cacheNames="products", key="#id", sync=true)
   public Product find(String id) { ... }
```

04 HOW IT WORKS, IN PLAIN ENGLISH

The abstraction imposes limitations, such as no unless with sync=true and a single cache. The exact Redis loader synchronization depends on the selected writer and framework version. Default nonlocking behavior should not be assumed to serialize misses across instances.

05 COMMON INTERVIEW TRAP

Verify concurrent cold misses on two pods rather than infer distributed safety.

06 FOLLOW-UP: HOW WOULD YOU MEASURE DUPLICATE LOADS?

Instrument database loader calls by cache key and run simultaneous cold requests on two pods. Count actual loads, cache misses, duration and peak concurrent loads; do not infer cross-pod locking from the annotation.

REDIS CACHE MANAGER CONFIGURATION

What does a locking RedisCacheWriter lock?

01 SHORT INTERVIEW ANSWER

In the 3.5 baseline, it coordinates selected multi-command cache operations at the cache level, not simply one entry-level business lock.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Nonlocking writer -> fewer requests
   Locking writer -> extra lock checks and possible waiting
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Writer locking can prevent command overlap but trades throughput for coordination. Its lifetime and scope need review. It is not proof that a database loader, cache entry or remote payment is protected by a robust business lock.

05 COMMON INTERVIEW TRAP

Cache-writer locking is not a replacement for a business invariant.

06 FOLLOW-UP: WHAT CONTENTION COULD A CACHE-WIDE LOCK CREATE?

A cache-wide writer lock can delay unrelated product keys while one operation clears or mutates the same named cache. That raises tail latency and limits throughput compared with narrowly scoped coordination.

08

CHAPTER 08 / 20

Cache consistency and invalidation

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q036
What is cache-aside?

Q037
Why prefer DB commit before cache eviction?

Q038
What are read-through, write-through and write-behind?

Q039
What is negative caching?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

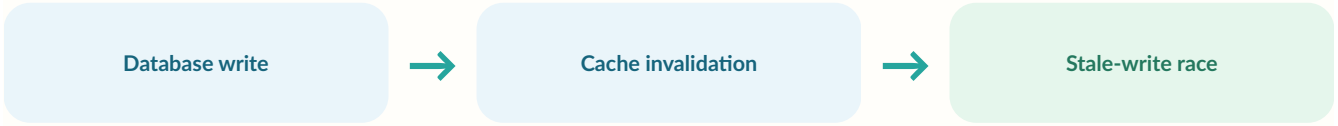
CACHE CONSISTENCY AND INVALIDATION

What is cache-aside?

01 SHORT INTERVIEW ANSWER

The application checks cache, loads the source on a miss, then populates cache.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
GET product:42 -> miss -> DB read -> SET with TTL
```

04 HOW IT WORKS, IN PLAIN ENGLISH

On writes, update the authoritative store and use a documented invalidation or update policy. Cache-aside allows miss races and temporary staleness. Define whether stale reads are acceptable and how they are repaired.

05 COMMON INTERVIEW TRAP

Redis does not automatically notice a SQL update.

06 FOLLOW-UP: WHICH WRITES BYPASS YOUR INVALIDATION PATH?

Admin scripts, batch jobs, SQL migrations, direct repository writes and other services can bypass application invalidation. List every writer to the authoritative data and define a shared notification or reconciliation path.

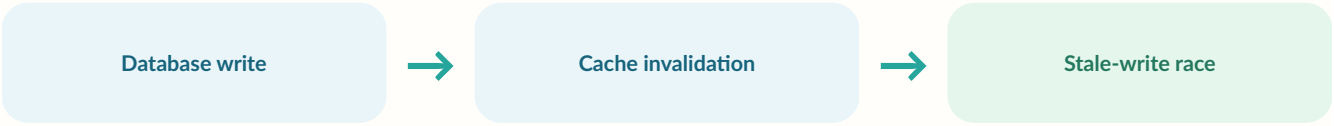
CACHE CONSISTENCY AND INVALIDATION

Why prefer DB commit before cache eviction?

01 SHORT INTERVIEW ANSWER

It avoids readers immediately repopulating cache from an uncommitted old database state, but it does not remove every race.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
DB update -> commit -> evict product:42
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A reader that loaded old data before commit can still write that data after eviction. Transaction-aware eviction helps align with commit, while version checks, bounded TTL or durable invalidation/reconciliation address remaining races.

05 COMMON INTERVIEW TRAP

Delete-after-commit alone is not strict consistency.

06 FOLLOW-UP: DRAW THE LATE STALE-LOADER INTERLEAVING.

Reader reads old DB value; writer updates DB and commits; writer evicts cache; reader finally SETs the old value with a fresh TTL. A version guard, source read or invalidation repair is needed to reject or bound it.

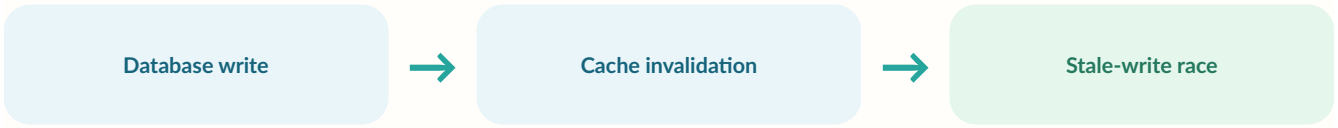
CACHE CONSISTENCY AND INVALIDATION

What are read-through, write-through and write-behind?

01 SHORT INTERVIEW ANSWER

They describe who loads values and when writes reach the authoritative backing store.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Read-through: cache layer loads miss
   Write-through: write also reaches backing store synchronously
   Write-behind: backing write delayed
```

04 HOW IT WORKS, IN PLAIN ENGLISH

These are architecture patterns, not all automatic RedisCacheManager capabilities. Write-behind needs durable buffering, ordering and failure recovery. A plain SET in Redis plus future database work is not automatically safe.

05 COMMON INTERVIEW TRAP

Delayed writes can disappear if the buffer is lost.

06 FOLLOW-UP: WHEN IS WRITE-BEHIND UNACCEPTABLE?

Write-behind is unacceptable when acknowledging an order, payment or ledger update before its only durable copy is committed. A process crash can lose queued writes, so use a durable queue/outbox or synchronous commit.

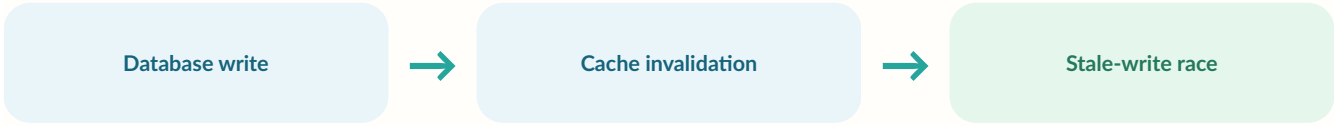
CACHE CONSISTENCY AND INVALIDATION

What is negative caching?

01 SHORT INTERVIEW ANSWER

It briefly caches a known absence or rejection to avoid repeated expensive source lookups.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
product:missing:42 -> ABSENT marker, TTL=10 seconds
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Distinguish absent data from a timeout or database outage. Keep a short lifetime and evict when the item is created. A null-caching strategy must match provider behavior and distinguish a miss from a stored negative value.

05 COMMON INTERVIEW TRAP

Caching infrastructure failures as absence can hide real data.

06 FOLLOW-UP: HOW WOULD YOU INVALIDATE A PREVIOUSLY ABSENT PRODUCT?

When creating a product, evict the previous negative entry for its ID or publish a reliable invalidation. Otherwise callers may keep receiving 'not found' until the short negative TTL expires.

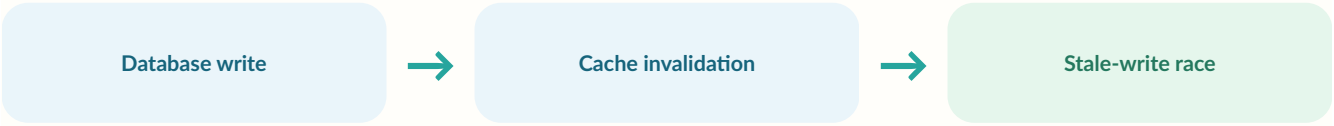
CACHE CONSISTENCY AND INVALIDATION

Can TTL guarantee fresh reads?

01 SHORT INTERVIEW ANSWER

No. TTL bounds the age of an entry from its expiry-setting operation, not freshness relative to every concurrent source update.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Old DB result loaded at T0
   Written into cache at T2 with TTL 60 seconds
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A delayed write can start a fresh TTL for old data. Exact consistency may require bypassing cache for critical reads or enforcing a versioned protocol with the authoritative store. Use TTL as one recovery bound, not a universal invariant.

05 COMMON INTERVIEW TRAP

Adding expiry does not solve all stale-write races.

06 FOLLOW-UP: WHICH ENDPOINT SHOULD READ DIRECTLY FROM THE SOURCE?

Read directly from the authoritative source for strict balances, authorization checks, scarce inventory decisions and other invariants that cannot tolerate stale data. Cache only a permitted projection or strictly versioned result.

09

CHAPTER 09 / 20

Stampedes, penetration and hot keys

Five focused questions, with concrete Redis behavior and safe failure boundaries.

Cold miss → Bounded loader → Fallback policy

What you will be able to explain

Q041
What is a cache stampede?

Q042
What is a cache avalanche?

Q043
What is cache penetration?

Q044
What is a hot key and why is Cluster not enough?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

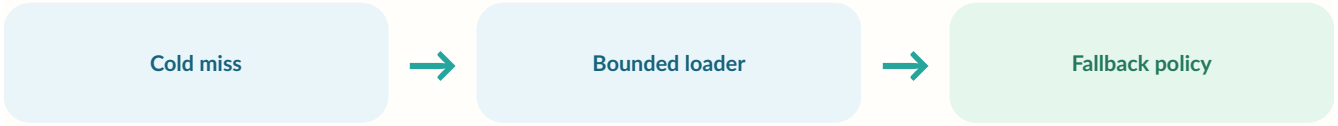
STAMPEDES, PENETRATION AND HOT KEYS

What is a cache stampede?

01 SHORT INTERVIEW ANSWER

Many requests miss the same key together and all load the backing service.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Hot key expires -> 1000 requests -> 1000 DB loads
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use bounded single-flight, carefully designed distributed loading coordination, refresh-ahead or stale-while-revalidate where valid. Add deadlines and cap concurrent loaders. Failure handling must not create endless waiting on a lost loader.

05 COMMON INTERVIEW TRAP

An unbounded lock wait just converts DB overload into request pileup.

06 FOLLOW-UP: HOW DOES THE SECOND CALLER BEHAVE WHEN THE LOADER FAILS?

If the shared loader fails, the second caller should receive a controlled failure or stale fallback after a bounded wait, not block forever. Clear an in-flight marker reliably and allow retry under a concurrency budget.

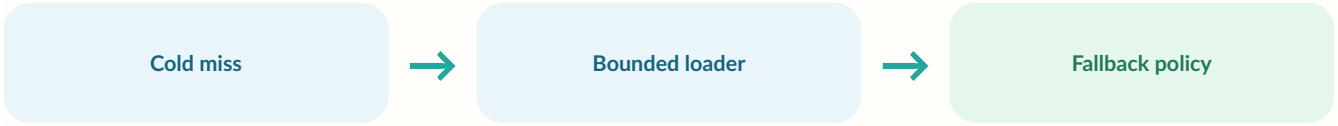
STAMPEDES, PENETRATION AND HOT KEYS

What is a cache avalanche?

01 SHORT INTERVIEW ANSWER

Many keys or an entire cache layer become unavailable together, creating a broad source-load surge.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
All product entries written with identical 300-second expiry
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use expiry jitter, staged warming and admission limits. A Redis outage also requires a fallback policy that respects database capacity. Independent key failures and broad dependency failure need different mitigation.

05 COMMON INTERVIEW TRAP

Blind fallback for every request can take down the database.

06 FOLLOW-UP: WHAT IS YOUR MAXIMUM PERMITTED FALLBACK CONCURRENCY?

Size fallback concurrency from database connection capacity and measured per-query latency, reserving headroom for normal traffic. Enforce the limit with a semaphore/bulkhead and return overload responses once it is reached.

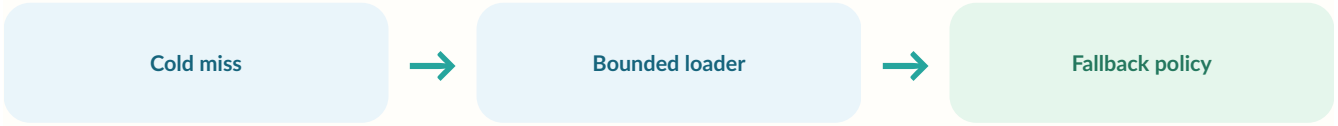
STAMPEDES, PENETRATION AND HOT KEYS

What is cache penetration?

01 SHORT INTERVIEW ANSWER

Repeated requests for nonexistent data bypass useful cache hits and repeatedly reach the source.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Requests for random invalid productIds -> repeated DB misses
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Validate identifiers, rate-limit abuse and consider negative caching. A Bloom filter can reject definite nonmembers, but false positives and update lifecycle must be managed; Bloom support may require server features/modules.

05 COMMON INTERVIEW TRAP

A probabilistic filter must not falsely reject valid new items through stale updates.

06 FOLLOW-UP: HOW DO YOU POPULATE IT BEFORE ENABLING IT?

Warm trusted keys via a bounded batch from authoritative data, or use a Bloom filter with a safe misses policy. Refresh the structure and handle false positives; never treat a filter as an authorization control.

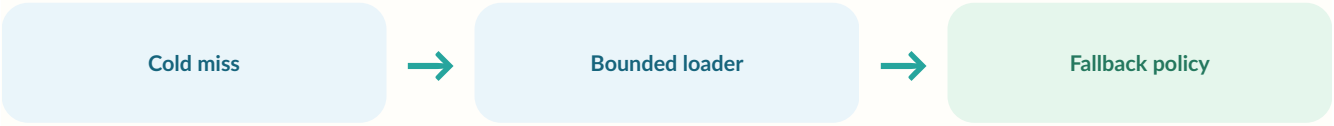
STAMPEDES, PENETRATION AND HOT KEYS

What is a hot key and why is Cluster not enough?

01 SHORT INTERVIEW ANSWER

A key receiving disproportionate traffic still maps to one slot/primary, so sharding other keys does not split its command work.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
90% of GETs -> sale:top-product -> one shard
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Reduce requests with local caching where staleness permits, replicate read-only derived data carefully, or redesign the aggregate. Read replicas introduce consistency tradeoffs. Measure key distribution without generating excessive metric labels.

05 COMMON INTERVIEW TRAP

Adding shards does not automatically distribute one key.

06 FOLLOW-UP: WHAT CORRECTNESS CHANGES IF A COUNTER IS SPLIT?

Splitting one logical counter can remove atomic admission and permit an overshoot until partial counts are merged. Use sharding only when approximate totals or a deliberate coordination protocol are acceptable.

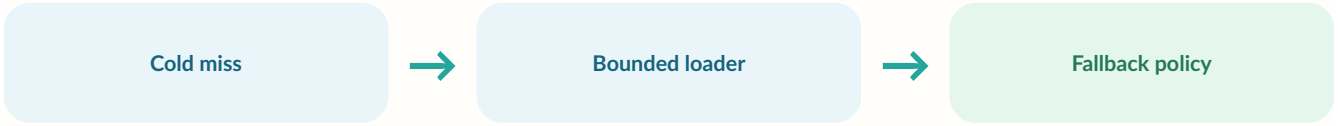
STAMPEDES, PENETRATION AND HOT KEYS

When is stale-while-revalidate useful?

01 SHORT INTERVIEW ANSWER

It serves a permitted stale value temporarily while a bounded refresh updates it asynchronously.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Soft deadline passed -> serve old catalog + trigger one refresh
   Hard deadline passed -> reviewed miss/failure behavior
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Keep soft and hard expiry explicit and ensure refresh coordination. This is application logic, not automatic behavior of plain @Cacheable. It fits read data with an accepted staleness budget and is unsuitable for arbitrary authorization or money decisions.

05 COMMON INTERVIEW TRAP

Stale data needs an agreed maximum age.

06 FOLLOW-UP: WHAT HAPPENS IF REFRESH KEEPS FAILING?

Serve only a bounded stale period; record refresh failures, retry with backoff and stop using data past maximum age. For sensitive data, fail closed or read the source instead of using stale content.

10

CHAPTER 10 / 20

Transactions, pipelines and scripts

Five focused questions, with concrete Redis behavior and safe failure boundaries.

Command sequence → Atomic boundary → Crash outcome

What you will be able to explain

Q046
What does Redis pipelining do?

Q047
What do MULTI and EXEC guarantee?

Q048
What is WATCH used for?

Q049
Why use Lua scripts for Redis operations?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

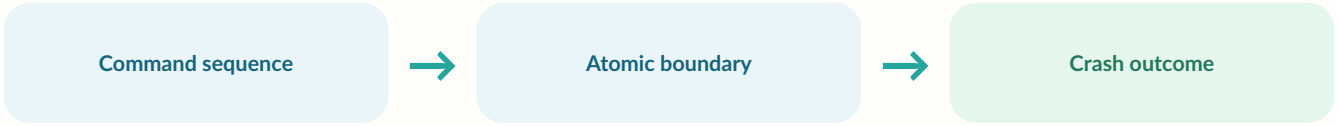
TRANSACTIONS, PIPELINES AND SCRIPTS

What does Redis pipelining do?

01 SHORT INTERVIEW ANSWER

It batches command transmission and response handling to reduce network round trips; it does not make commands atomic.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Pipeline: GET a, GET b, GET c -> batched round trip
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use bounded batches to avoid excessive response memory and latency. Other clients can interleave commands. Spring executePipelined collects results; serialization and connection usage need review. A timeout can leave uncertain command outcomes.

05 COMMON INTERVIEW TRAP

A pipeline is not a transaction.

06 FOLLOW-UP: HOW WOULD YOU PROVE INTERLEAVING CAN STILL HAPPEN?

Run another client's SET while the pipeline issues multiple reads; the replies can reflect different instants. Use tracing/barriers or a controlled concurrent test; pipelining reduces round trips, not isolation.

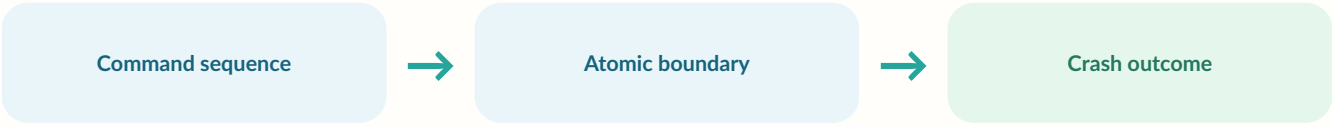
TRANSACTIONS, PIPELINES AND SCRIPTS

What do MULTI and EXEC guarantee?

01 SHORT INTERVIEW ANSWER

They queue commands and execute the queued sequence without other clients interleaving, but do not provide general rollback of runtime errors.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SET wrongType not-an-integer
   MULTI
   SET a 1
   INCR wrongType
   EXEC
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Errors detected during queuing can abort the transaction. A runtime error in one queued command does not undo successfully executed commands. Reads inside the queue cannot drive immediate conditional Java logic using their eventual values.

05 COMMON INTERVIEW TRAP

Redis transactions are not SQL rollback semantics.

06 FOLLOW-UP: HOW WOULD YOU AVOID PARTIAL LOGICAL UPDATES FROM BAD TYPES?

Validate types and values before writing, preferably within a short Lua script that checks all preconditions first. MULTI/EXEC executes queued commands in order but a runtime command error does not roll back earlier commands.

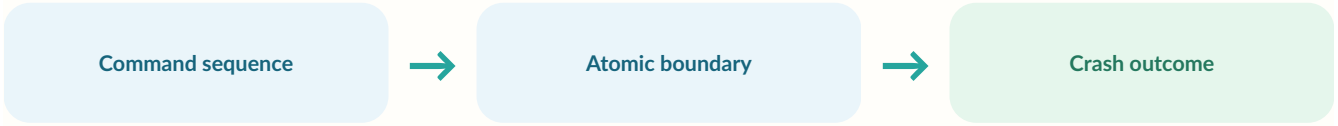
TRANSACTIONS, PIPELINES AND SCRIPTS

What is WATCH used for?

01 SHORT INTERVIEW ANSWER

WATCH provides optimistic conflict detection: EXEC aborts if watched keys changed before execution.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
WATCH stock:42
   GET stock:42
   MULTI; SET stock:42 9; EXEC
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use one connection for the sequence and bounded retries after conflicts. Expiration behavior and supported commands should be checked for the server version. Avoid an unbounded spin loop under contention.

05 COMMON INTERVIEW TRAP

WATCH does not lock the key against other writers.

06 FOLLOW-UP: WHEN WOULD LUA BE SIMPLER?

Lua is simpler for a small atomic check-and-update such as increment + first-expiry or compare-token + delete. WATCH works for optimistic, client-side read/compute/update transactions that can retry on conflicts.

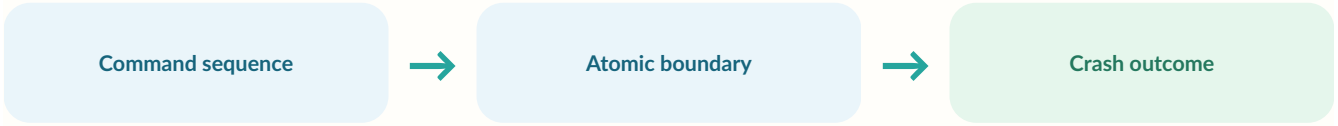
TRANSACTIONS, PIPELINES AND SCRIPTS

Why use Lua scripts for Redis operations?

01 SHORT INTERVIEW ANSWER

A script can combine read/check/write logic atomically on Redis without application round trips between steps.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
if GET lock equals token then DEL lock end
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Scripts must be short, bounded and declare accessed keys appropriately. They block competing command execution while running and do not roll back all prior writes after a runtime error. Validate inputs/types before mutation. In Cluster, required keys normally share a slot.

05 COMMON INTERVIEW TRAP

Atomic execution is not unlimited safe computation.

06 FOLLOW-UP: WHAT HAPPENS WHEN A SCRIPT LOOPS FOR TOO LONG?

A long-running script blocks other server commands and increases latency. Keep scripts bounded by data size, monitor slowlog/latency, and never scan an unbounded keyspace inside one script.

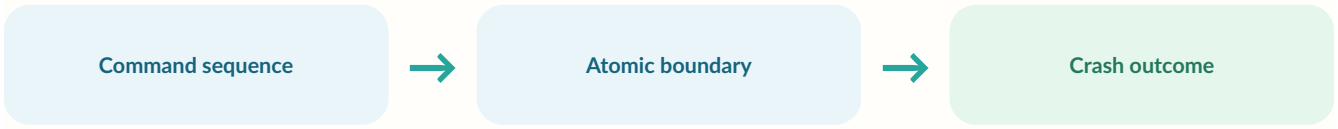
TRANSACTIONS, PIPELINES AND SCRIPTS

How does Spring Data support Redis transactions?

01 SHORT INTERVIEW ANSWER

Use a same-connection SessionCallback for explicit MULTI/EXEC; optional template transaction support can bind connections to Spring transactions.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
template.execute(new SessionCallback<>() { /* MULTI, queued ops, EXEC */ });
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Ordinary template calls may not share the required connection unless coordinated. Enabling transaction support is a configuration choice. Redis and SQL still do not become one general atomic distributed transaction merely because Spring synchronizes them.

05 COMMON INTERVIEW TRAP

@Transactional does not automatically make every template call transactional.

06 FOLLOW-UP: HOW WOULD YOU RECOVER SQL COMMIT FOLLOWED BY REDIS FAILURE?

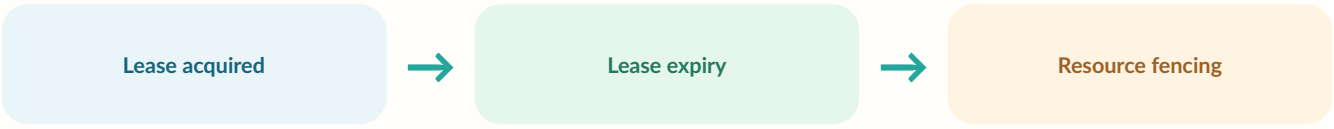
Treat the SQL commit as authoritative and use a durable outbox/change event to retry Redis invalidation or rebuild. This is reconciliation across two systems; @Transactional alone cannot atomically commit SQL and Redis.

11

CHAPTER 11 / 20

Distributed locks and fencing

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q051
How do you acquire a basic Redis lease safely?

Q052
Why must release compare the ownership token?

Q053
What is a fencing token?

Q054
Does automatic lease renewal solve every lock problem?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

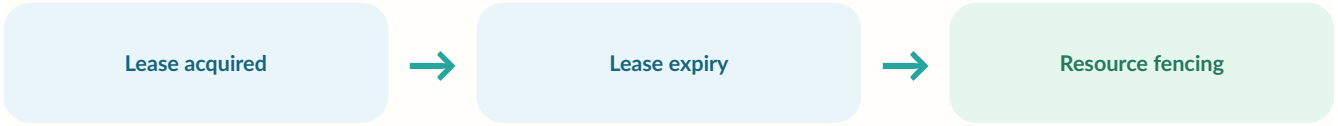
DISTRIBUTED LOCKS AND FENCING

How do you acquire a basic Redis lease safely?

01 SHORT INTERVIEW ANSWER

Use one conditional SET with a unique ownership token and an expiry.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SET lock:job:42 unique-token NX PX 10000
```

04 HOW IT WORKS, IN PLAIN ENGLISH

NX makes acquisition conditional and PX sets the lease in the same operation. The token distinguishes owners. This is a basic single-Redis lease with assumptions, not a proof of mutual exclusion through every failover or pause.

05 COMMON INTERVIEW TRAP

SETNX followed later by EXPIRE has a crash gap.

06 FOLLOW-UP: WHAT IF THE HOLDER PAUSES LONGER THAN TEN SECONDS?

After the TTL expires, another worker may obtain the lease while the paused holder resumes. The old holder must not be allowed to perform an external protected effect; use fencing/version checks at that resource.

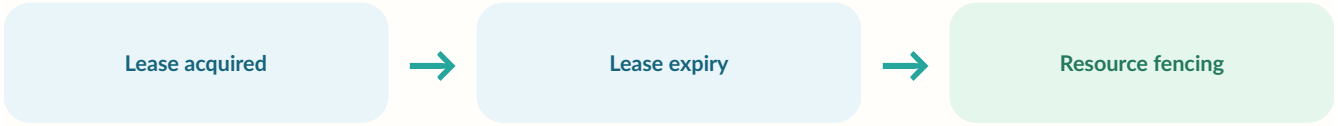
DISTRIBUTED LOCKS AND FENCING

Why must release compare the ownership token?

01 SHORT INTERVIEW ANSWER

A previous holder must not delete a lease now owned by a different caller.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
A's lease expires -> B acquires
   A resumes -> unconditional DEL would remove B's lease
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use an atomic compare-and-delete script or a server-version-supported equivalent. GET followed by DEL has its own race. The token helps safe release but does not stop a paused holder from modifying an external resource.

05 COMMON INTERVIEW TRAP

Token-safe unlock is not fencing.

06 FOLLOW-UP: HOW WOULD YOU PROTECT THE DATABASE FROM STALE A?

Give each lease acquisition a monotonic fencing number and have the database reject a write with an older number. Compare-and-delete protects B's Redis key; only resource-side enforcement blocks stale A's business write.

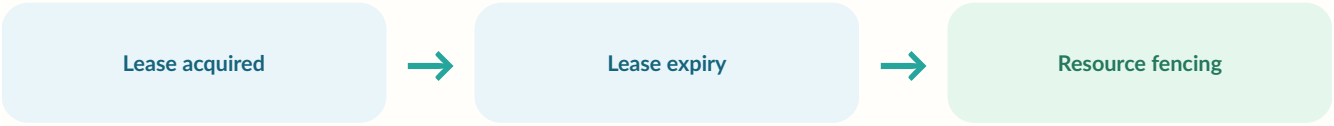
DISTRIBUTED LOCKS AND FENCING

What is a fencing token?

01 SHORT INTERVIEW ANSWER

It is an increasing generation attached to work that the protected resource uses to reject stale holders.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
A token=10 pauses; B token=11 writes
   DB rejects later write carrying token=10
```

04 HOW IT WORKS, IN PLAIN ENGLISH

The downstream resource must enforce token order atomically. Token allocation and survival through failover must meet the required guarantee; an asynchronously replicated INCR is not automatically a failover-proof fencing service.

05 COMMON INTERVIEW TRAP

Generating a number without checking it at the resource does nothing.

06 FOLLOW-UP: WHAT IF THE TOKEN ALLOCATOR LOSES ITS LATEST VALUE?

If the allocator can reset to a previous value after failover, stale holders may appear new. Allocate tokens using a durable monotonic authority or enforce an equivalent authoritative row/version constraint.

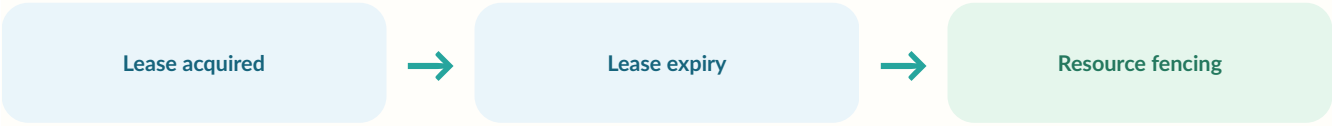
DISTRIBUTED LOCKS AND FENCING

Does automatic lease renewal solve every lock problem?

01 SHORT INTERVIEW ANSWER

No. Renewal reduces accidental expiry during active work but cannot eliminate long pauses, lost connectivity or stale external effects.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Heartbeat renew fails -> lease can expire
   Worker must stop or have its writes fenced
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Renew only if the ownership token still matches. Bound work duration and detect ownership loss, while acknowledging a paused process may resume too late to stop safely. Robust business invariants remain necessary.

05 COMMON INTERVIEW TRAP

A watchdog is not a proof of exactly-once processing.

06 FOLLOW-UP: HOW WOULD YOU TEST A LONG GC PAUSE?

Pause a worker beyond its lease TTL using a test barrier, then let a second worker acquire and write. Resume the old worker; assert the downstream resource rejects its stale token even if auto-renew was configured.

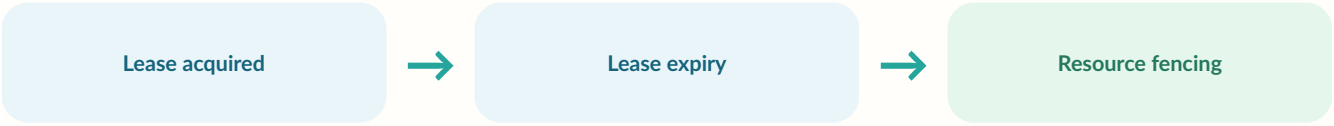
DISTRIBUTED LOCKS AND FENCING

What is the failover risk with a single replicated Redis lock?

01 SHORT INTERVIEW ANSWER

An acknowledged lock may not reach the replica before failover, allowing another client to acquire it on the new primary.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
A acquires on primary -> primary fails before replication
   Replica promoted -> B sees no lease and acquires
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Redis replication is normally asynchronous. Alternatives require explicit safety assumptions, resource fencing or a stronger coordination system. Do not claim Sentinel or Redis Cluster alone makes this lease linearizable.

05 COMMON INTERVIEW TRAP

High availability and lock safety are different properties.

06 FOLLOW-UP: WHICH INVARIANT CAN A DATABASE CONSTRAINT ENFORCE INSTEAD?

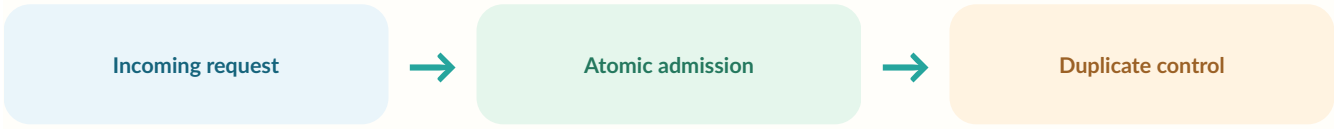
A database unique constraint or conditional row update can enforce one reservation per item or nonnegative stock. Redis lease ownership alone cannot guarantee those invariants across failover and pauses.

12

CHAPTER 12 / 20

Rate limiting and idempotency

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q056
How does a fixed-window rate limiter work?

Q057
How does a sliding-window limiter differ?

Q058
What is a token bucket?

Q059
Should a rate limiter fail open or fail closed?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

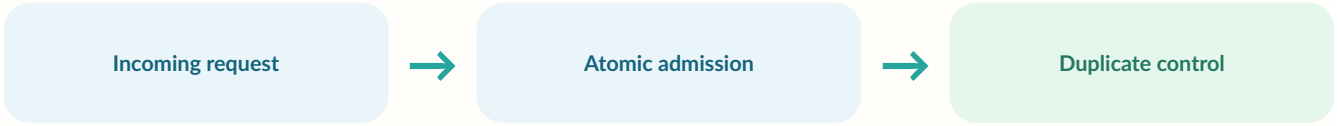
RATE LIMITING AND IDEMPOTENCY

How does a fixed-window rate limiter work?

01 SHORT INTERVIEW ANSWER

It counts requests in a time bucket and permits requests up to a threshold.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Key: limit:user-42:minute-123
   INCR; allow if count <= 100
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Counter creation and expiry should follow one atomic policy. Fixed windows allow a burst around boundaries, such as 100 requests at the end and 100 at the start. Decide which events count and return retry information.

05 COMMON INTERVIEW TRAP

Separate INCR/EXPIRE can create immortal counters.

06 FOLLOW-UP: WHEN IS A BOUNDARY BURST UNACCEPTABLE?

A two-window boundary burst may admit nearly twice the nominal rate in a short real interval. Use a sliding window or token bucket when that burst would overwhelm the protected service or break a policy.

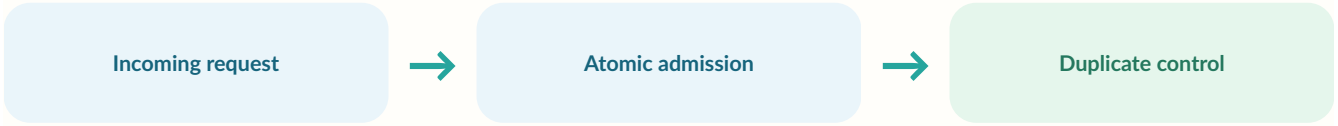
RATE LIMITING AND IDEMPOTENCY

How does a sliding-window limiter differ?

01 SHORT INTERVIEW ANSWER

It evaluates recent activity across a moving interval rather than resetting at one bucket boundary.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Sorted set: requestId -> timestamp
   Remove older than window; count; conditionally add
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Combine prune/count/admission in one script. Unique members prevent requests sharing a timestamp from overwriting each other. Bound key growth and expiry. More precise algorithms require more work and state than fixed windows.

05 COMMON INTERVIEW TRAP

Millisecond timestamp alone may not be a unique member.

06 FOLLOW-UP: HOW WOULD YOU LIMIT SCRIPT COST?

Limit tracked events per identity, remove obsolete members each call, and cap Lua work/time. For very high volume, consider a bounded-memory approximation or token bucket instead of unbounded sorted sets.

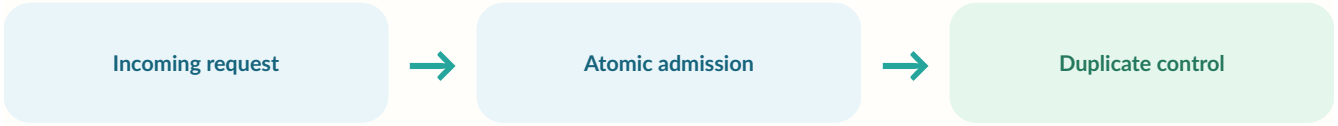
RATE LIMITING AND IDEMPOTENCY

What is a token bucket?

01 SHORT INTERVIEW ANSWER

It refills a bounded token balance over time and consumes tokens for admitted work, allowing a controlled burst.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Capacity=20 tokens; refill=5 tokens/second
   Request cost=1 token
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Store balance and last-refill time consistently, and define time source and precision. An atomic script avoids concurrent overspending. Distinguish per-user limits from per-tenant and global resource admission.

05 COMMON INTERVIEW TRAP

A user-supplied identity can be rotated to evade the limit.

06 FOLLOW-UP: WHAT KEY PROTECTS A SHARED DOWNSTREAM CAPACITY?

Use a key scoped to the actual shared resource, such as provider:payment-api or tenant+provider if isolation is required. A per-pod key does not protect aggregate downstream capacity.

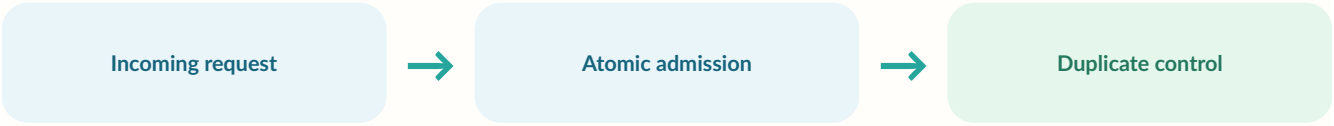
RATE LIMITING AND IDEMPOTENCY

Should a rate limiter fail open or fail closed?

01 SHORT INTERVIEW ANSWER

Choose explicitly from abuse risk, availability and business impact; there is no universally correct policy.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Catalog reads: bounded local fallback may be acceptable
   Payment attempts: reviewed stricter admission policy
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A Redis error must not silently become a cache miss or a fresh zero counter without analysis. Local emergency limits may preserve partial protection but differ across pods. Monitor degraded-mode entry and exit.

05 COMMON INTERVIEW TRAP

Fail-open can amplify the very overload Redis was controlling.

06 FOLLOW-UP: WHICH ENDPOINT NEEDS A DIFFERENT POLICY?

Reject or tightly limit calls when Redis fails for login attempts, payment protection or abuse-sensitive endpoints; permit bounded fallback for noncritical browsing. Define different policies per endpoint and measure failure impact.

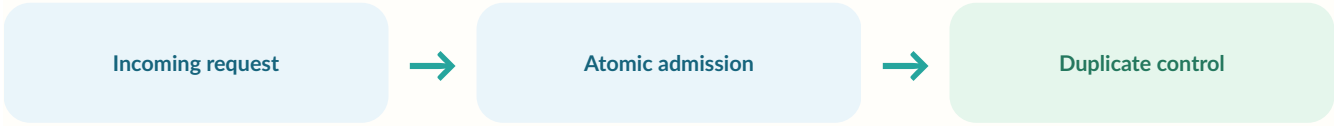
RATE LIMITING AND IDEMPOTENCY

Can SET NX with a TTL guarantee one business effect?

01 SHORT INTERVIEW ANSWER

No. A deduplication marker and a separate business effect leave a crash gap, and expiry permits future repeats.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Set marker -> crash before effect: skipped work
   Effect -> crash before marker: duplicate work
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use a durable unique constraint and effect transaction for local DB work, or the external provider's idempotency contract. Redis can assist coordination but should not promise atomicity with an unrelated store.

05 COMMON INTERVIEW TRAP

A cache-only dedup flag is weak protection for payments.

06 FOLLOW-UP: HOW LONG MUST AN IDEMPOTENCY RECORD SURVIVE REPLAY?

Keep the request ID/result at least as long as any possible client retry, queue replay and delayed duplicate. Make the authoritative business effect idempotent too, since Redis TTL expiry may outlive in-flight work.

13

CHAPTER 13 / 20

Redis Cluster

Five focused questions, with concrete Redis behavior and safe failure boundaries.

Key hash slot → Primary + replica → Client routing

What you will be able to explain

Q061
How does Redis Cluster distribute keys?

Q062
What do hash tags do?

Q063
Why do CROSSSLOT errors occur?

Q064
How do MOVED and ASK differ?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

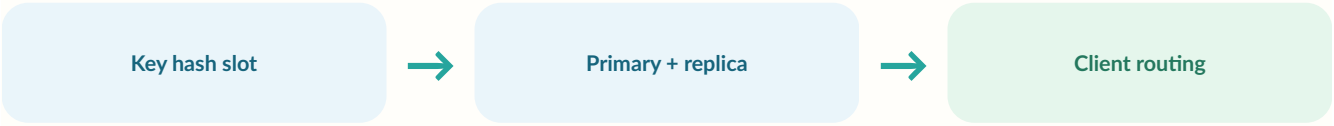
REDIS CLUSTER

How does Redis Cluster distribute keys?

01 SHORT INTERVIEW ANSWER

Redis Cluster maps keys to 16384 hash slots distributed among primaries.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
slot = CRC16(relevant key bytes) modulo 16384
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Cluster-aware clients discover topology and route commands. Slots are not separate physical databases or a count of nodes. Adding nodes requires slot movement; replicas support availability rather than independent primary write shards.

05 COMMON INTERVIEW TRAP

Six nodes do not automatically mean six write primaries.

06 FOLLOW-UP: HOW WOULD THREE PRIMARIES AND THREE REPLICAS BE ARRANGED?

Use three primary nodes covering the 16,384 hash slots and one replica per primary, placed on different failure domains where possible. Each replica follows one primary; plan promotion and capacity during failures.

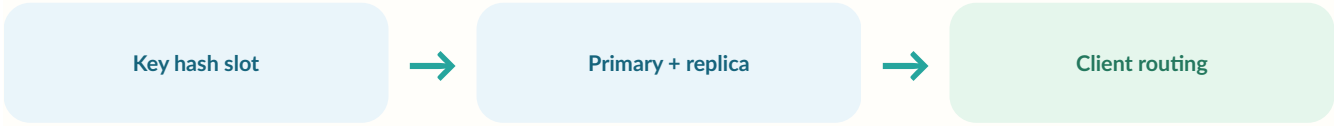
REDIS CLUSTER

What do hash tags do?

01 SHORT INTERVIEW ANSWER

A valid {...} substring controls slot hashing so related keys can be colocated.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
order:{42}:state
   order:{42}:lock
```

04 HOW IT WORKS, IN PLAIN ENGLISH

These keys share a slot, enabling suitable multi-key operations. Key-tag syntax follows Redis rules, so empty or multiple brace patterns need care. Putting every key under one common tag defeats distribution.

05 COMMON INTERVIEW TRAP

Tenant-wide colocation can create a hot slot.

06 FOLLOW-UP: WHICH KEYS TRULY NEED ATOMIC MULTI-KEY ACCESS?

Only keys that must be modified atomically in one multi-key command/script should share a hash tag, such as wallet:{account42}:balance and wallet:{account42}:holds. Do not force unrelated tenants into one hot slot.

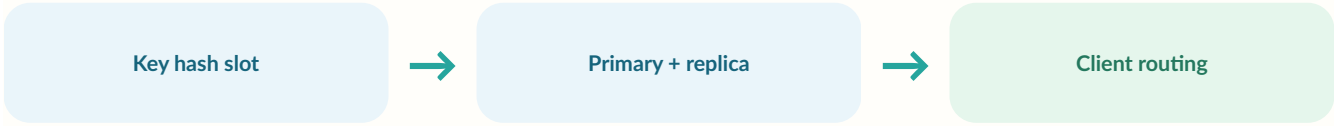
REDIS CLUSTER

Why do CROSSSLOT errors occur?

01 SHORT INTERVIEW ANSWER

The operation requires keys to share a slot but the supplied keys map to different slots.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
MULTI/script with order:42 and balance:7 -> possible CROSSSLOT
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use deliberate hash tags or redesign the operation. Some client APIs decompose operations into separate node commands, but that does not preserve server-side cross-key atomicity. Check the chosen command and client implementation.

05 COMMON INTERVIEW TRAP

A client-side multiGet convenience does not imply a global transaction.

06 FOLLOW-UP: HOW WOULD YOU PRESERVE A CROSS-AGGREGATE INVARIANT?

Keep cross-aggregate invariants in a transactional authoritative store or use an explicit saga/reservation protocol. Cluster key colocation alone cannot create atomicity across independent owners or failure domains.

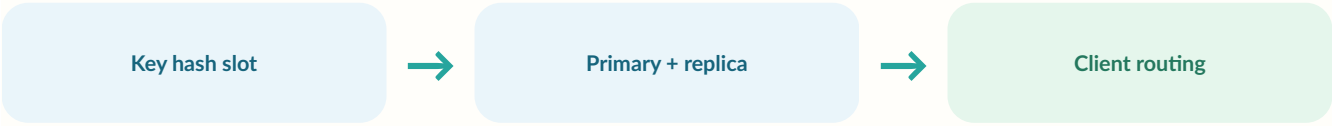
REDIS CLUSTER

How do MOVED and ASK differ?

01 SHORT INTERVIEW ANSWER

MOVED indicates the slot's routing owner; ASK is a temporary redirection used during migration with special handling.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
MOVED -> update routing knowledge
   ASK -> redirected operation with ASKING as required
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A cluster-aware driver handles redirects. Topology refresh helps after resharding/failover, but network reachability to advertised nodes remains essential. A standalone client pointed at one cluster node is insufficient.

05 COMMON INTERVIEW TRAP

Retrying only against the original node can loop or fail.

06 FOLLOW-UP: WHAT ADDRESSES MUST AN APPLICATION POD REACH?

Pods must reach every advertised Cluster node address/port returned by MOVED or ASK, not only the initial bootstrap endpoint. Test DNS, TLS and routing during resharding and failover.

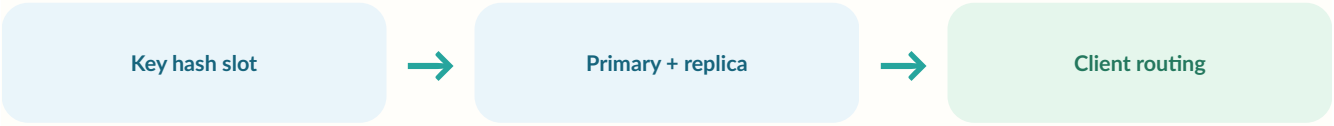
REDIS CLUSTER

How do you configure Boot for a Redis Cluster?

01 SHORT INTERVIEW ANSWER

Configure cluster nodes and driver refresh policy appropriate to the deployment instead of standalone-only host assumptions.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
spring.data.redis.cluster.nodes=node1:6379,node2:6379,node3:6379
   spring.data.redis.lettuce.cluster.refresh.adaptive=true
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Seed nodes bootstrap discovery; the client then reaches relevant advertised nodes. Review redirect limits, TLS names, credentials and periodic/adaptive refresh. Cluster supports database zero rather than SELECT-based numbered DB isolation.

05 COMMON INTERVIEW TRAP

A reachable seed does not prove every slot owner is reachable.

06 FOLLOW-UP: HOW WOULD YOU TEST A RESHARDING ROLLOUT?

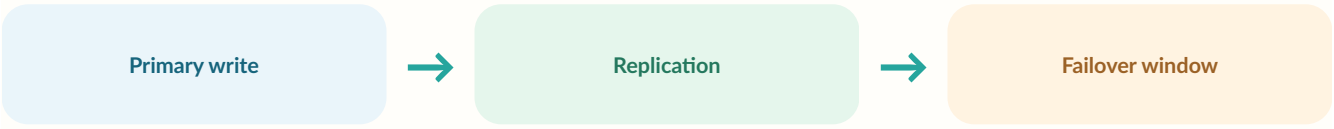
Run a controlled slot migration during traffic while tracking MOVED/ASK handling, error rates and latency. Verify that keys, scripts and client topology refresh recover without duplicate business effects.

14

CHAPTER 14 / 20

Replication, Sentinel and durability

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q066
What does Redis replication provide?

Q067
What does Sentinel do?

Q068
How do Sentinel and Cluster differ?

Q069
How do RDB and AOF persistence differ?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

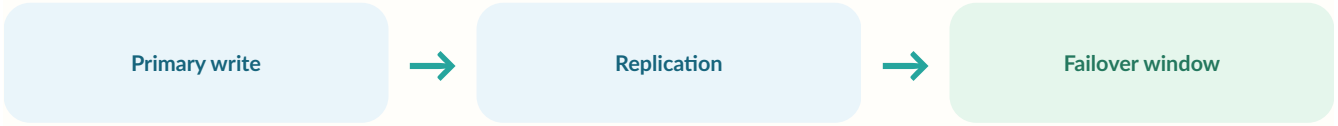
REPLICATION, SENTINEL AND DURABILITY

What does Redis replication provide?

01 SHORT INTERVIEW ANSWER

It copies a primary's data to replicas, normally asynchronously, for availability and optional read scaling.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Primary acknowledges write -> replica may catch up afterward
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A replica read can be stale. Failover can lose recent acknowledged writes that never reached the promoted replica. Choose read routes and acceptable loss explicitly for sessions, counters and authoritative state.

05 COMMON INTERVIEW TRAP

Replication is not automatically synchronous durability.

06 FOLLOW-UP: WHICH DATA MUST NEVER BE SERVED FROM A STALE REPLICA?

Do not serve balances, payment status or authorization changes from a replica when staleness is unacceptable. Replica reads are eventually consistent and need an explicit permitted-staleness policy.

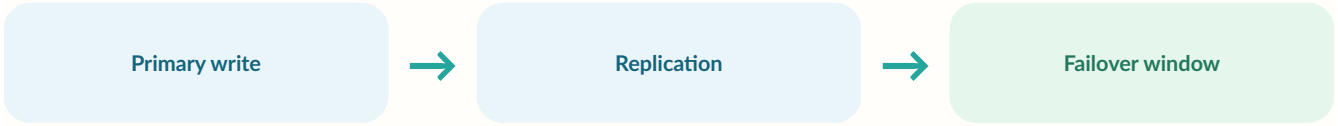
REPLICATION, SENTINEL AND DURABILITY

What does Sentinel do?

01 SHORT INTERVIEW ANSWER

Sentinel monitors a nonclustered primary/replica deployment and coordinates discovery and automatic failover.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Sentinel quorum -> detect failure -> promote replica
   Application client -> discover current primary
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Sentinel does not shard the keyspace. Configure the logical master name, Sentinel addresses and credentials correctly. Quorum and failover authorization involve different requirements; multiple independent failure domains matter.

05 COMMON INTERVIEW TRAP

One Sentinel is not a robust high-availability design.

06 FOLLOW-UP: HOW WOULD CLIENTS RECOVER AFTER PROMOTION?

The Sentinel-aware client discovers the new master and reconnects/retries within deadlines. Test writes during promotion and reconcile uncertain outcomes; in-flight commands are not magically atomic across failover.

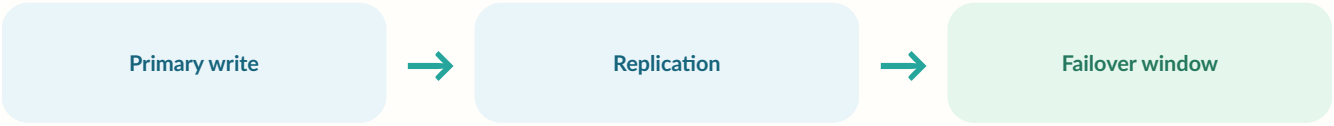
REPLICATION, SENTINEL AND DURABILITY

How do Sentinel and Cluster differ?

01 SHORT INTERVIEW ANSWER

Sentinel provides failover for a primary-replica topology; Cluster combines sharding with its own failover mechanisms.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Need more total write/memory capacity -> evaluate Cluster
   Need HA for one primary dataset -> evaluate Sentinel
```

04 HOW IT WORKS, IN PLAIN ENGLISH

The choice also depends on managed-service capabilities and consistency needs. Cluster adds multi-key restrictions and topology-aware routing. Neither removes asynchronous replication loss windows automatically.

05 COMMON INTERVIEW TRAP

Sentinel is not the coordinator Redis Cluster requires.

06 FOLLOW-UP: WHICH APPLICATION COMMANDS COMPLICATE A CLUSTER MIGRATION?

Cross-slot Lua/MULTI operations, broad multi-key commands, key scanning, server-side moves and connection routing need review for Cluster. Sentinel provides failover for a nonsharded primary, not distributed hash slots.

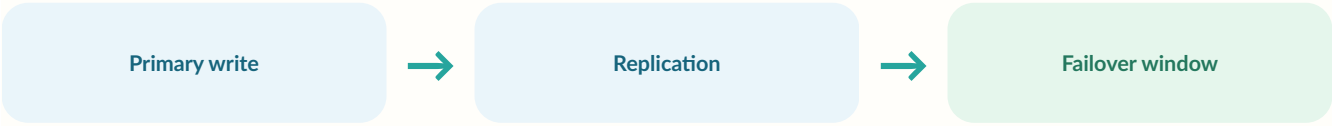
REPLICATION, SENTINEL AND DURABILITY

How do RDB and AOF persistence differ?

01 SHORT INTERVIEW ANSWER

RDB stores snapshots; AOF records write operations for replay under its configured persistence policy.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
RDB -> snapshot recovery point
   AOF appendfsync everysec -> typical small recent-loss window
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Snapshot intervals, fsync behavior, rewrite and recovery configuration determine durability and resource cost. An every-second setting is not an absolute no-loss guarantee under all failures. Test backup restore, not only file creation.

05 COMMON INTERVIEW TRAP

Persistence on one disk is not a backup strategy.

06 FOLLOW-UP: WHAT IS YOUR RECOVERY-POINT OBJECTIVE?

Set an RPO based on the worst business loss you can tolerate and test snapshots, AOF policy, replication and restore time against it. State that acknowledged writes may still be lost under some crash/failover patterns.

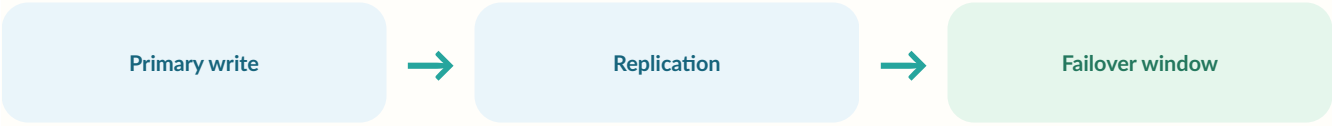
REPLICATION, SENTINEL AND DURABILITY

Does WAIT guarantee strong consistency or no data loss?

01 SHORT INTERVIEW ANSWER

No. WAIT can request acknowledgment from replicas for prior writes, but does not turn Redis into a strongly consistent system.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Write -> WAIT 1 1000 -> inspect returned replica count
```

04 HOW IT WORKS, IN PLAIN ENGLISH

The returned count indicates observed replication acknowledgments, not a universal persistence or failover guarantee. It must apply to the relevant connection/write sequence. Review persistence acknowledgment options separately for supported server versions.

05 COMMON INTERVIEW TRAP

Waiting for one replica is not a proof every failure preserves the write.

06 FOLLOW-UP: WHAT HAPPENS IF WAIT TIMES OUT AFTER THE WRITE SUCCEEDED?

The write may already exist even if WAIT times out. Do not blindly resend a non-idempotent increment; read back state or retry with a stable operation ID and an authoritative deduplication protocol.

15

CHAPTER 15 / 20

Memory, eviction and performance

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q071
How do expiration and eviction differ?

Q072
How do allkeys, volatile and noeviction policies differ?

Q073
Why are big keys dangerous?

Q074
Why prefer SCAN to KEYS for production inspection?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

MEMORY, EVICTION AND PERFORMANCE

How do expiration and eviction differ?

01 SHORT INTERVIEW ANSWER

Expiration removes logically expired keys; eviction removes keys under configured memory pressure policy.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
TTL remaining=60 seconds
   maxmemory pressure -> key can still be evicted
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use memory policies consistent with the dataset. A cache can tolerate eviction; a lock, session or authoritative record may not. Separate workloads when policy conflict matters.

05 COMMON INTERVIEW TRAP

TTL is not a minimum guaranteed residency period.

06 FOLLOW-UP: SHOULD LOCK KEYS SHARE AN ALLKEYS-LRU CACHE?

No. Critical lease/lock state should not share a memory pool using allkeys eviction, because the lock could disappear before TTL. Separate instances or use a policy/capacity model that does not evict live coordination keys.

MEMORY, EVICTION AND PERFORMANCE

How do allkeys, volatile and noeviction policies differ?

01 SHORT INTERVIEW ANSWER

Allkeys policies can choose any eligible key; volatile policies restrict candidates to expiring keys; noeviction rejects applicable writes when memory is exhausted.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
allkeys-lru; volatile-lru; allkeys-lfu; noeviction
```

04 HOW IT WORKS, IN PLAIN ENGLISH

LRU/LFU selection is approximate rather than a perfect global ranking. A volatile policy with no suitable candidates cannot simply free arbitrary permanent keys. Monitor write errors as well as eviction counts.

05 COMMON INTERVIEW TRAP

noeviction does not mean Redis has unlimited memory.

06 FOLLOW-UP: WHAT HAPPENS TO A NEW SET AT THE LIMIT?

Under noeviction, a memory-increasing SET can fail with an OOM error; with eviction policies, eligible keys may be removed. Test application handling and never rely on eviction to preserve a critical invariant.

MEMORY, EVICTION AND PERFORMANCE

Why are big keys dangerous?

01 SHORT INTERVIEW ANSWER

Large values or collections can increase network cost, command latency, memory use and deletion/recovery work.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
One hash with millions of fields -> HGETALL returns a huge response
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Prefer bounded access and pagination-like iteration where appropriate. Measure payload size and collection cardinality. UNLINK can move memory freeing away from the immediate delete path, but it does not make huge reads inexpensive.

05 COMMON INTERVIEW TRAP

One key can be too large even with a low total key count.

06 FOLLOW-UP: HOW DO YOU IDENTIFY IT SAFELY?

Use SCAN to find candidate keys gradually and MEMORY USAGE or sampled size tools with strict rate limits. Avoid dumping huge values or running KEYS in production; correlate with slow commands and bandwidth.

MEMORY, EVICTION AND PERFORMANCE

Why prefer SCAN to KEYS for production inspection?

01 SHORT INTERVIEW ANSWER

KEYS traverses the keyspace in one blocking operation; SCAN incrementally iterates it.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SCAN 0 MATCH shop:prod:* COUNT 1000
   Repeat using returned cursor until cursor=0
```

04 HOW IT WORKS, IN PLAIN ENGLISH

COUNT is a hint, not an exact result size. A live scan can return duplicates and is not a snapshot. In Cluster, scan the required node scope with a cluster-aware strategy. Bound cleanup batches and tolerate changing data.

05 COMMON INTERVIEW TRAP

One SCAN call does not enumerate the entire database.

06 FOLLOW-UP: HOW WOULD YOU MAKE CLEANUP SAFE UNDER CONCURRENT WRITES?

Use cursor-based SCAN with throttling and narrow prefixes, verify each key's expected version before deletion, and tolerate duplicates/new writes. For business data, prefer a versioned namespace switch over live broad deletion.

MEMORY, EVICTION AND PERFORMANCE

How do you diagnose Redis latency?

01 SHORT INTERVIEW ANSWER

Correlate command cost, server CPU/memory, network, payloads, connection waits, persistence and client timeouts.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Slow command duration != entire client round-trip latency
```

04 HOW IT WORKS, IN PLAIN ENGLISH

SLOWLOG measures command execution time rather than all network wait. Use command statistics and approved latency tooling alongside application metrics. A long script or large response can affect many callers.

05 COMMON INTERVIEW TRAP

Increasing timeouts hides symptoms without fixing the bottleneck.

06 FOLLOW-UP: WHICH EVIDENCE DISTINGUISHES NETWORK FROM COMMAND COST?

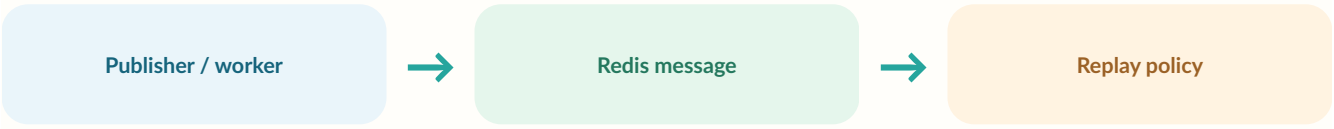
Compare server SLOWLOG/LATENCY data with client timers for connect, acquire, encode, send and response. Low server execution time with high client round-trip time points to network, queueing or client-side contention.

16

CHAPTER 16 / 20

Sessions, Pub/Sub and Streams

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q076
How does Spring Session use Redis?

Q077
What delivery guarantee does Redis Pub/Sub provide?

Q078
How do Redis Streams differ from Pub/Sub?

Q079
What is the pending entries list in Streams?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

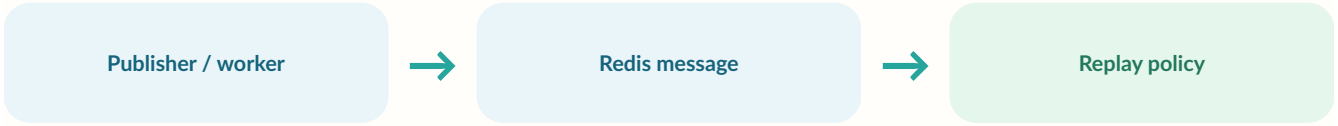
SESSIONS, PUB/SUB AND STREAMS

How does Spring Session use Redis?

01 SHORT INTERVIEW ANSWER

It can store HTTP session state centrally so multiple application instances can access a user's session.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
spring-session-data-redis dependency
   session cookie -> shared server-side session state
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Choose the appropriate Session repository and configuration for the Boot version. Session expiration, events, serialization and security are separate from product caching. Redis failure can affect login continuity directly.

05 COMMON INTERVIEW TRAP

Shared sessions do not justify caching arbitrary sensitive objects.

06 FOLLOW-UP: HOW DO YOU ROTATE SESSIONS AFTER AUTHENTICATION?

After successful login, rotate the session ID through the supported session-fixation protection and persist context correctly. Apply suitable timeout, cookie flags and logout/revocation behavior for the actual Spring Session deployment.

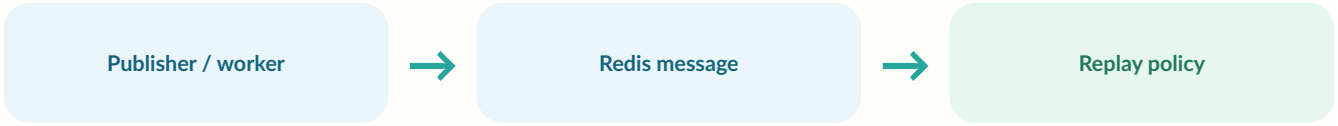
SESSIONS, PUB/SUB AND STREAMS

What delivery guarantee does Redis Pub/Sub provide?

01 SHORT INTERVIEW ANSWER

Redis Pub/Sub is at-most-once delivery to currently connected subscribers, without retained replay history.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Subscriber disconnected during PUBLISH -> message missed
```

04 HOW IT WORKS, IN PLAIN ENGLISH

It fits best-effort notifications when loss is acceptable or a separate repair path exists. A listener reconnect does not recover missed messages. Do not use it as the only durable business event transport.

05 COMMON INTERVIEW TRAP

Pub/Sub is not a persistent queue.

06 FOLLOW-UP: HOW WOULD AN L1 CACHE REPAIR A MISSED INVALIDATION?

Use short local TTLs, version checks on reads, periodic reconciliation or invalidation fanout with recovery. Pub/Sub messages can be missed while an app is disconnected, so a missed notice needs an independent repair path.

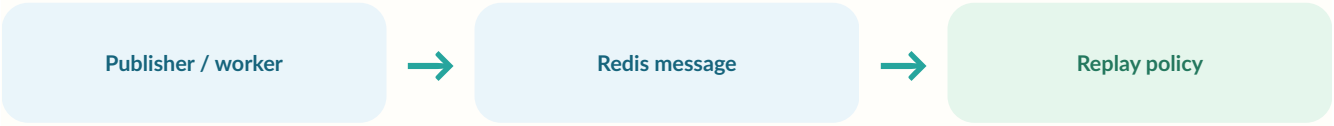
SESSIONS, PUB/SUB AND STREAMS

How do Redis Streams differ from Pub/Sub?

01 SHORT INTERVIEW ANSWER

Streams retain entries and support consumer groups, progress acknowledgment and pending-entry recovery.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
XADD orders * eventId e-42
   XREADGROUP GROUP workers c1 STREAMS orders >
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Retention/trimming and persistence policy still matter. Consumer groups distribute work and maintain pending entries. Business effects can repeat when unacknowledged entries are recovered, so idempotency is required.

05 COMMON INTERVIEW TRAP

A stored stream is not infinite history.

06 FOLLOW-UP: WHAT HAPPENS WHEN PENDING ENTRIES ARE TRIMMED AWAY?

If trimming removes pending stream entries, consumers can retain delivery metadata without a recoverable payload. Monitor retention against processing lag, claim pending items safely and define poison/dead-letter recovery.

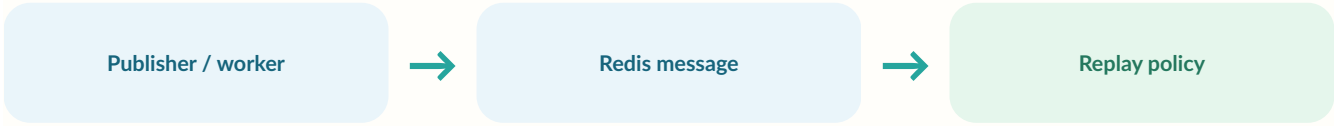
SESSIONS, PUB/SUB AND STREAMS

What is the pending entries list in Streams?

01 SHORT INTERVIEW ANSWER

It tracks entries delivered to a group consumer that have not yet been acknowledged.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
XPENDING orders workers
   XACK orders workers <entry-id>
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A failed consumer's entries need a claim/recovery policy, such as XAUTOCLAIM on supported versions. Acknowledge after the required durable effect. Examine idle time, delivery count and poison records during recovery.

05 COMMON INTERVIEW TRAP

Reading only new entries does not automatically finish all old pending work.

06 FOLLOW-UP: HOW DO YOU DETECT AN ABANDONED CONSUMER?

Inspect XPENDING and idle duration/delivery attempts by consumer, then use XAUTOCLAIM on supported versions to transfer eligible abandoned work. Acknowledge only after the durable effect succeeds.

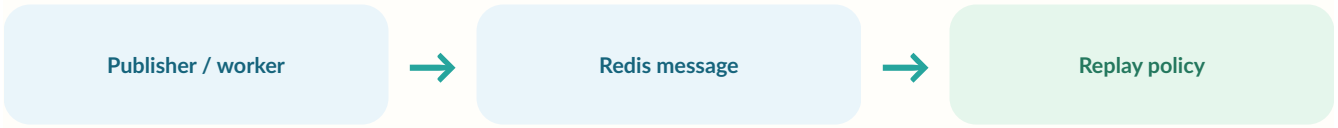
SESSIONS, PUB/SUB AND STREAMS

When should you choose Kafka instead of Redis Streams?

01 SHORT INTERVIEW ANSWER

Choose from required retention, throughput, ecosystem, ordering, persistence and operational isolation rather than a universal winner.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Large durable event platform -> evaluate Kafka
   Redis-centric bounded work stream -> evaluate Streams
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Kafka and Redis Streams have different replication, storage and consumer mechanics. Benchmark realistic workloads and failure recovery. Shared Redis caching and stream state can compete for memory unless isolated deliberately.

05 COMMON INTERVIEW TRAP

API similarity does not imply equivalent durability.

06 FOLLOW-UP: WHAT RECOVERY EVIDENCE WOULD DETERMINE YOUR CHOICE?

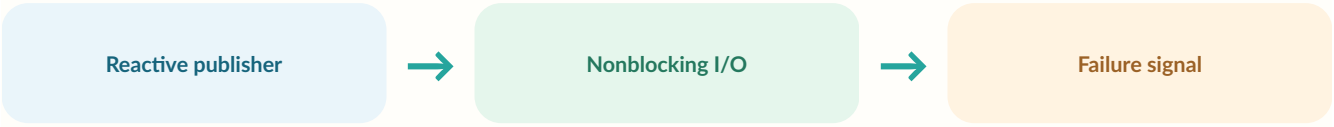
Choose by measured history retention, replay/rebuild needs, consumer scaling, throughput and operational guarantees. Test a crash/replay and confirm the oldest required event is still available before choosing either technology.

17

CHAPTER 17 / 20

Reactive Redis and asynchronous boundaries

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q081

What does ReactiveRedisTemplate provide?

Q082

Why avoid block() on a reactive request path?

Q083

How should a reactive cache-aside miss be handled?

Q084

Are retries always safe for Redis writes?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

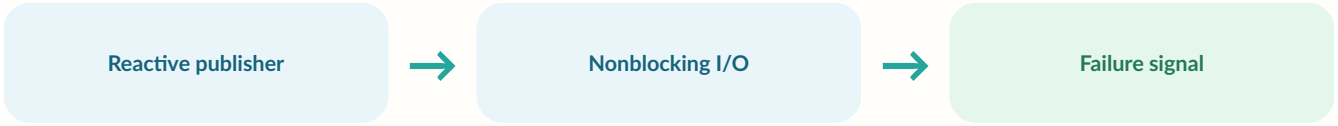
REACTIVE REDIS AND ASYNCHRONOUS BOUNDARIES

What does ReactiveRedisTemplate provide?

01 SHORT INTERVIEW ANSWER

It exposes Redis operations through Reactor types with reactive driver support.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Mono<String> value = reactiveStrings.opsForValue().get("product:42");
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Operations generally occur on subscription; compose the result into the application response. Configure serialization in the reactive context explicitly. Reactive access is a client execution model, not a faster Redis command algorithm.

05 COMMON INTERVIEW TRAP

Constructing a Mono alone does not prove a write occurred.

06 FOLLOW-UP: WHO SUBSCRIBES IN A WEBFLUX CONTROLLER?

The WebFlux framework subscribes to a controller-returned Mono/Flux to drive request processing. Application code should compose publishers rather than manually calling subscribe inside the endpoint.

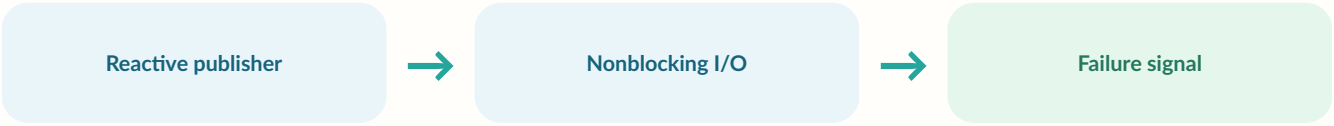
REACTIVE REDIS AND ASYNCHRONOUS BOUNDARIES

Why avoid block() on a reactive request path?

01 SHORT INTERVIEW ANSWER

Blocking waits can stall an event-loop thread and undermine nonblocking throughput.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
return redis.opsForValue().get(key)
       .switchIfEmpty(loadFromReactiveSource());
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Keep the source truly nonblocking or isolate unavoidable blocking work on a reviewed scheduler with bounded resources. Also make loading lazy when constructing the fallback would perform work eagerly.

05 COMMON INTERVIEW TRAP

A method returning Mono can still block internally.

06 FOLLOW-UP: HOW WOULD YOU IDENTIFY A BLOCKING DATABASE CALL?

Use BlockHound where suitable, thread/CPU profiling, traces and bounded load tests to find JDBC or other blocking work on event-loop threads. Replace with reactive I/O or explicitly isolate unavoidable blocking calls.

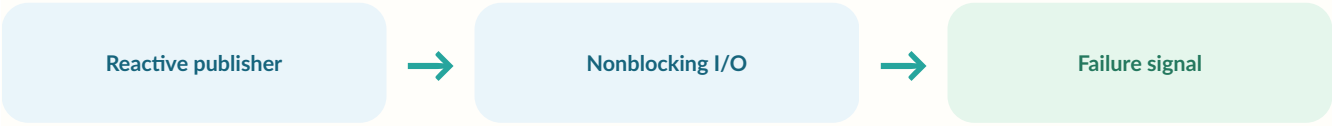
REACTIVE REDIS AND ASYNCHRONOUS BOUNDARIES

How should a reactive cache-aside miss be handled?

01 SHORT INTERVIEW ANSWER

Compose a lazy source lookup, then cache only a valid returned value under an explicit write-failure policy.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
GET Mono -> switchIfEmpty(Mono.defer(loadAndCache))
```

04 HOW IT WORKS, IN PLAIN ENGLISH

An empty Mono and a Redis error are different outcomes. Decide whether a cache-write error should fail a source-backed read or return valid source data. Guard concurrent miss load and cancellation behavior.

05 COMMON INTERVIEW TRAP

onErrorResume(ignored -> empty()) can hide a real dependency failure.

06 FOLLOW-UP: WHICH EXCEPTIONS ARE SAFE TO TREAT AS CACHE UNAVAILABLE?

Treat only classified connection/availability failures as optional for a noncritical cache. Do not turn deserialization, authorization, programming or data-integrity failures into silent cache misses.

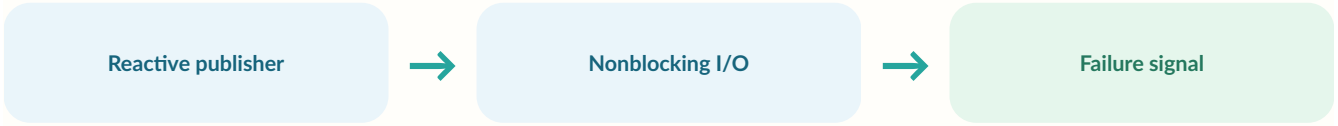
REACTIVE REDIS AND ASYNCHRONOUS BOUNDARIES

Are retries always safe for Redis writes?

01 SHORT INTERVIEW ANSWER

No. A timeout can occur after the server applied the command; repeating nonidempotent operations can duplicate effects.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
INCR applied -> response lost -> retry INCR -> count increased twice
```

04 HOW IT WORKS, IN PLAIN ENGLISH

GET is naturally repeatable; SET with the same value may still alter TTL; INCR and append operations have effect ambiguity. Use stable request identity and an atomic dedup/effect design when required. Bound backoff and avoid retry storms.

05 COMMON INTERVIEW TRAP

Transport failure is not proof the command did nothing.

06 FOLLOW-UP: HOW WOULD YOU MAKE A COUNTER REQUEST IDEMPOTENT?

Deduplicate the operation ID and apply the increment in one atomic step, with enough deduplication retention for retries. A blind retry after an uncertain response can count the same request twice.

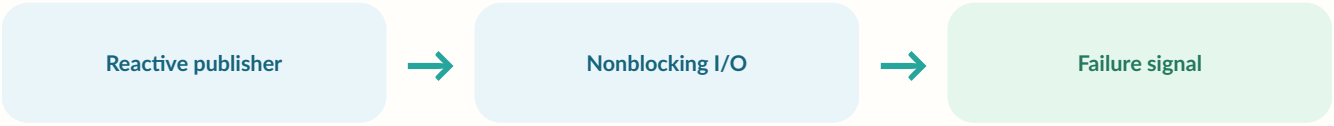
REACTIVE REDIS AND ASYNCHRONOUS BOUNDARIES

How should timeouts and cancellation be interpreted?

01 SHORT INTERVIEW ANSWER

They bound caller waiting, but do not necessarily roll back or cancel a command already accepted by Redis.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Client times out at 100 ms
   Server may finish SET at 120 ms
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Distinguish connection failure, enqueue delay and unknown response outcome where the client exposes them. Maintain operation identity for correctness-critical writes. Cancellation of a subscriber is not a general compensating operation on Redis.

05 COMMON INTERVIEW TRAP

Timeout does not mean absent effect.

06 FOLLOW-UP: WHAT SHOULD AN API REPORT FOR AN UNCERTAIN WRITE?

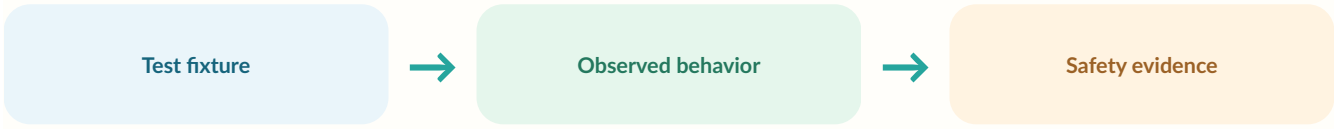
Report that the outcome is unknown or accepted-for-reconciliation, with a stable request ID for lookup. A timeout does not prove the write failed; avoid a false success or unconditional repeat.

18

CHAPTER 18 / 20

Testing, observability and security

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q086
What should a Redis unit test cover?

Q087
How do you test TTL without brittle sleeps?

Q088
Which metrics show whether caching helps?

Q089
How should Redis access be secured?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

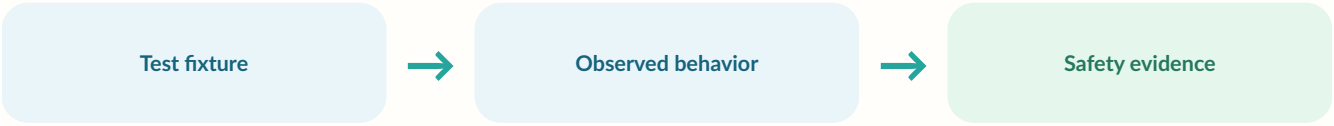
TESTING, OBSERVABILITY AND SECURITY

What should a Redis unit test cover?

01 SHORT INTERVIEW ANSWER

Test key construction, policies and business decisions without pretending mocked Redis proves server behavior.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Tenant included in key; TTL policy chosen; invalid input rejected
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Integration tests should cover serialization, expiry, Lua, transactions and the configured cache proxy. A mocked get/set cannot validate atomicity, replication or Cluster routing. Select a server fixture matching the tested feature.

05 COMMON INTERVIEW TRAP

A mock counter does not prove concurrent increments are safe.

06 FOLLOW-UP: WHICH TEST NEEDS AN ACTUAL REDIS INSTANCE?

An actual Redis integration test is needed to verify TTL, serialization bytes, Lua atomic behavior, pipelining and Cluster routing. A mocked unit test can check key construction and app-level fallback logic only.

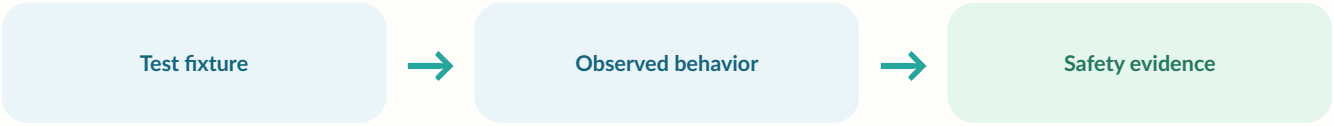
TESTING, OBSERVABILITY AND SECURITY

How do you test TTL without brittle sleeps?

01 SHORT INTERVIEW ANSWER

Use a short test expiry and bounded polling for the expected state transition, checking the initial TTL as well.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Set with TTL -> assert PTTL positive
   Await until GET absent within a bounded timeout
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Use unique keys or isolated fixtures. Test overwrite behavior and absence/no-expiry sentinel values. Expiration timing should be asserted with reasonable tolerance rather than millisecond equality. Cluster/failover behavior needs additional fixtures.

05 COMMON INTERVIEW TRAP

A passing timeout-only test may miss a key created without expiry.

06 FOLLOW-UP: WHAT SHOULD FAILURE DIAGNOSTICS INCLUDE?

Use a short explicit TTL and poll with bounded await until the key is absent instead of sleeping an exact number of milliseconds. On failure, include observed TTL, clock budget, key namespace and Redis endpoint details without secrets.

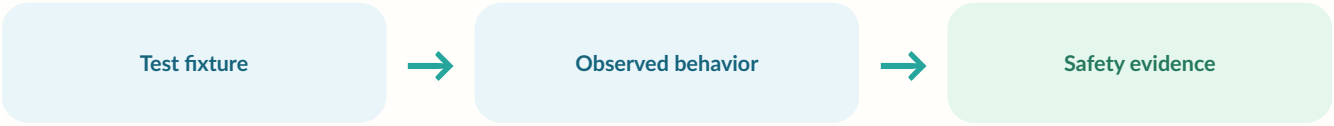
TESTING, OBSERVABILITY AND SECURITY

Which metrics show whether caching helps?

01 SHORT INTERVIEW ANSWER

Measure hits, misses, load latency, source load, Redis errors, evictions, memory and end-to-end latency together.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Hit ratio high + stale data complaints -> correctness issue
   Hit ratio low + DB saturation -> load issue
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Statistics can be local to a cache manager and must be enabled/configured where supported. Add business freshness evidence. Keep metric labels bounded; individual customer IDs belong in protected logs, not metric tag sets.

05 COMMON INTERVIEW TRAP

Hit ratio alone is not a success criterion.

06 FOLLOW-UP: WHAT FRESHNESS SLO WOULD YOU TRACK?

Track an upper-bound age of cached data or percentage of reads meeting the approved freshness budget, alongside hit rate. A 99% hit rate with stale prices can be worse than a lower hit rate with correct values.

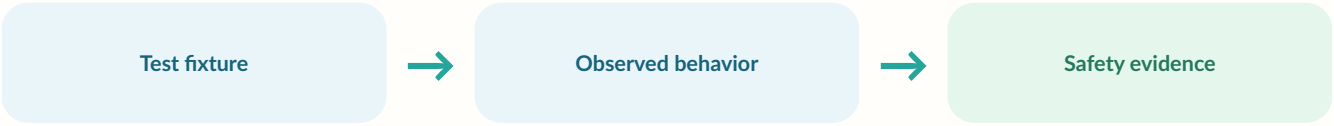
TESTING, OBSERVABILITY AND SECURITY

How should Redis access be secured?

01 SHORT INTERVIEW ANSWER

Use appropriate TLS, authentication, ACLs, network isolation and controlled administration privileges.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Application principal -> permitted key namespace + commands
   Admin principal -> separate controlled operations
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Redis ACL users support finer privileges than one shared unrestricted credential. Store secrets safely, test rotation and preserve hostname verification. Restrict scripts, destructive commands and diagnostic data exposure according to the actual role.

05 COMMON INTERVIEW TRAP

Key prefixes do not enforce permissions without ACL policy.

06 FOLLOW-UP: WHICH COMMANDS DOES YOUR CACHE CLIENT REALLY NEED?

Grant only the needed command categories/keys, for example GET/SET/DEL on a namespaced cache; avoid dangerous administrative commands. Use Redis ACLs, TLS, secret rotation and private network isolation.

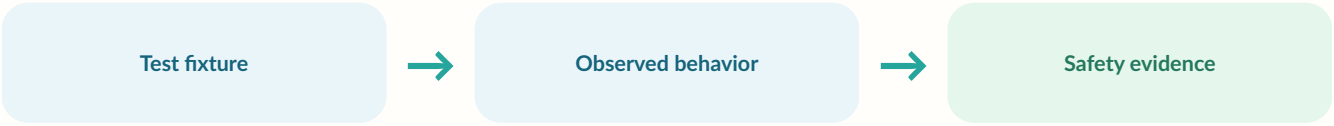
TESTING, OBSERVABILITY AND SECURITY

How should Redis health affect readiness and liveness?

01 SHORT INTERVIEW ANSWER

Readiness reflects the service's ability to fulfill its role; liveness should avoid restarting a recoverable process for every remote outage.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Optional cache outage -> bounded source fallback
   Mandatory session store outage -> reviewed readiness failure
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Define dependency timeouts and degraded-mode capacity. Cache, lock, rate-limiter and session roles need different policies. A restart storm can worsen reconnection pressure without repairing Redis.

05 COMMON INTERVIEW TRAP

A healthy connection does not prove every slot or command is usable.

06 FOLLOW-UP: WHAT OPERATION WOULD YOUR HEALTH EVIDENCE REPRESENT?

Liveness should test whether the process can make progress; readiness should reflect whether it can safely serve its promised endpoints. For optional caching, Redis down may not justify marking the entire app unready if bounded source fallback works.

19

CHAPTER 19 / 20

Production troubleshooting scenarios

Five focused questions, with concrete Redis behavior and safe failure boundaries.

Incident symptom → Root cause → Safe recovery

What you will be able to explain

Q091

@Cacheable runs every time. What do you inspect?

Q092

A cached update is stale after eviction. What could have happened?

Q093

The app gets WRONGTYPE errors. What is likely?

Q094

CROSSSLOT appears after moving to Cluster. What changed?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

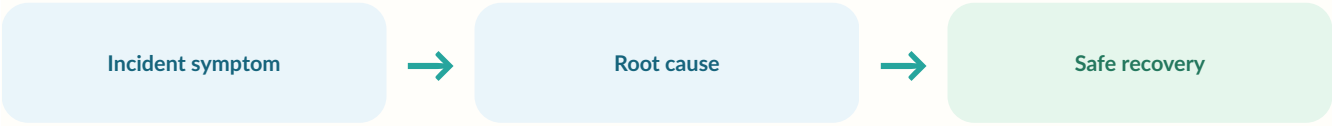
PRODUCTION TROUBLESHOOTING SCENARIOS

@Cacheable runs every time. What do you inspect?

01 SHORT INTERVIEW ANSWER

Check caching enablement, proxy path, cache manager, keys, expiry and write failures.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
this.find(id) -> bypass
   new Service() -> unmanaged instance
   Different tenant/argument key -> different entry
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Inspect the actual cache key and serializer, not a guessed CLI key. Verify the configured manager is Redis rather than a local provider. A method's invocation counter across calls can demonstrate hits without relying on timing.

05 COMMON INTERVIEW TRAP

Seeing Redis configured does not prove that annotation uses it.

06 FOLLOW-UP: HOW WOULD YOU PROVE THE MANAGER AND PROXY ARE CORRECT?

Invoke the service through a Spring-managed proxy, check @EnableCaching, bean type and the effective RedisCacheManager. Assert the second call skips the source and inspect the exact cache key/TTL in Redis.

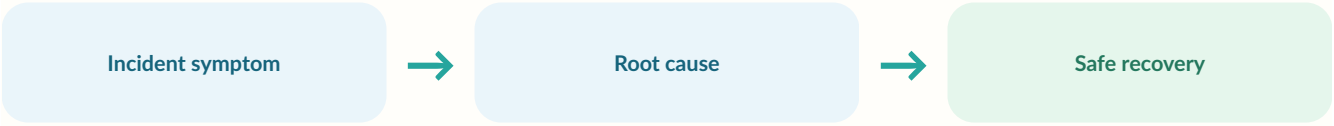
PRODUCTION TROUBLESHOOTING SCENARIOS

A cached update is stale after eviction. What could have happened?

01 SHORT INTERVIEW ANSWER

An earlier reader may have loaded old state and populated it after the writer evicted the key.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Reader DB old -> writer commits + evicts -> reader SET old
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Reproduce this interleaving with test barriers. Consider versioned values/conditional updates, authoritative critical reads and durable invalidation with reconciliation. Do not call this proof Redis lost the delete.

05 COMMON INTERVIEW TRAP

Repeating deletion once more is not a universal race-free protocol.

06 FOLLOW-UP: HOW DO YOU BOUND OR REJECT A LATE STALE WRITE?

Use versioned values with compare-before-write, source version checks, or authoritative reads for strict fields. Otherwise shorten TTL and reconcile. A delayed loader must not overwrite an entry representing a newer DB version.

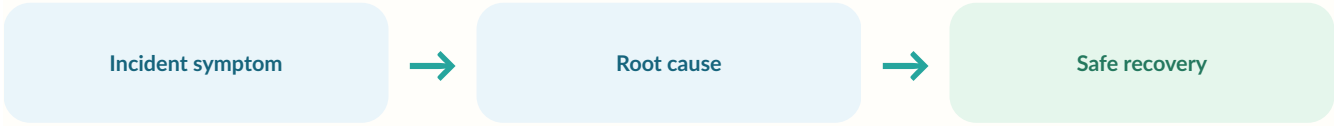
PRODUCTION TROUBLESHOOTING SCENARIOS

The app gets WRONGTYPE errors. What is likely?

01 SHORT INTERVIEW ANSWER

The operation targets a key holding a different Redis data type than expected.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
SET profile:42 JSON
   HGET profile:42 name -> WRONGTYPE
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Check namespace collisions, old deployments and data migrations using TYPE and safe inspection. Serialization mistakes can coexist with type mistakes but are distinct. Repair only the intended keyspace and rollout contract.

05 COMMON INTERVIEW TRAP

FLUSHALL is not a diagnostic fix.

06 FOLLOW-UP: HOW DO YOU MIGRATE THE STRUCTURE WITHOUT DELETING UNRELATED DATA?

Write a new key namespace/type, adapt readers to both versions and migrate incrementally. Delete only owned, verified v1 keys after rollback expires; do not flush unrelated Redis databases.

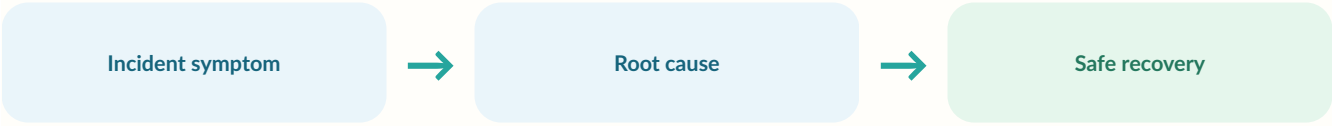
PRODUCTION TROUBLESHOOTING SCENARIOS

CROSSSLOT appears after moving to Cluster. What changed?

01 SHORT INTERVIEW ANSWER

A multi-key operation or script now spans different slots, even if it worked on standalone Redis.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Two independent keys -> standalone allowed
   Cluster multi-key atomic operation -> colocation required
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Audit scripts, transactions, multi-key commands and cache-clearing strategies. Hash tags can help specific related keys, but broad tags can create skew. Client decomposition can change atomicity, so verify semantics.

05 COMMON INTERVIEW TRAP

Making every key share one tag defeats sharding.

06 FOLLOW-UP: WHICH INVARIANT TRULY REQUIRES COLOCATION?

Only state that must change together in one Redis multi-key command requires key colocation. Business invariants that cross independent aggregates should be handled at their transactional owner, not forced into a single hash slot.

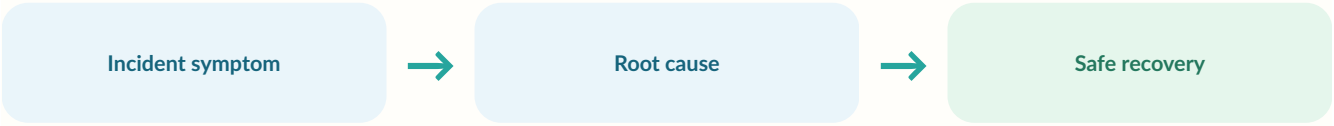
PRODUCTION TROUBLESHOOTING SCENARIOS

Redis goes down and the database also overloads. Why?

01 SHORT INTERVIEW ANSWER

Unbounded cache fallback or synchronized retries redirected too much traffic to the source.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Cache miss/error -> every request calls DB -> pool exhausted
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Introduce bounded admission, circuit policy, controlled retries and graceful responses. Warm or refresh gradually after recovery. Test outage behavior under load rather than assuming the source can absorb full cache traffic.

05 COMMON INTERVIEW TRAP

Fail-open is not free availability.

06 FOLLOW-UP: HOW WOULD YOU CHOOSE THE FALLBACK CONCURRENCY LIMIT?

Measure spare DB capacity and the acceptable error rate; reserve most connections for normal traffic and enforce a semaphore/bulkhead for cache misses. Add circuit breaking and return fast overload responses when that budget is exhausted.

20

CHAPTER 20 / 20

Senior architecture and design challenges

Five focused questions, with concrete Redis behavior and safe failure boundaries.



What you will be able to explain

Q096

How would you design a two-level product cache?

Q097

Can Redis locks prevent inventory overselling by themselves?

Q098

How do you prevent cross-tenant cache leakage?

Q099

How do you roll out a new cache serialization format?

Interview habit: Always separate the normal Redis operation from the business effect and the crash window.

SENIOR ARCHITECTURE AND DESIGN CHALLENGES

How would you design a two-level product cache?

01 SHORT INTERVIEW ANSWER

Use bounded local caching plus Redis with an explicit version/invalidation and maximum-staleness policy.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
L1 Caffeine -> L2 Redis -> authoritative DB
   Invalidation event -> both layers; missed event -> expiry/repair
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Pub/Sub invalidation can be missed while a pod is disconnected. TTL or version validation provides a repair bound. Ensure cold-start load and serializer changes are handled. For strict freshness, some reads must consult the authoritative path.

05 COMMON INTERVIEW TRAP

L1 can remain stale even after the Redis key was evicted.

06 FOLLOW-UP: HOW WOULD YOU TEST A MISSED INVALIDATION?

Disable the invalidation subscription for one node or drop an event, update the product, then check how quickly it converges. Verify L1 maximum TTL/version reads and ensure unauthorized data is never exposed during delay.

SENIOR ARCHITECTURE AND DESIGN CHALLENGES

Can Redis locks prevent inventory overselling by themselves?

01 SHORT INTERVIEW ANSWER

Not reliably across every failure. Enforce the inventory invariant at the authoritative mutation boundary.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
UPDATE stock SET available=available-1
   WHERE item_id=:id AND available>0;
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A conditional SQL update or appropriately designed authoritative atomic Redis operation can enforce the actual invariant. A separate expiring lease is only coordination and may fail during pause/failover. Define reservation expiry, release and reconciliation.

05 COMMON INTERVIEW TRAP

Mutual exclusion assumptions are weaker than a resource-enforced invariant.

06 FOLLOW-UP: WHAT IF THE SAME RESERVATION REQUEST IS DELIVERED TWICE?

Use a stable reservation ID and an authoritative unique constraint or conditional inventory update so repeated delivery returns the existing outcome. Redis lease acquisition is an optimization, not the only oversell protection.

SENIOR ARCHITECTURE AND DESIGN CHALLENGES

How do you prevent cross-tenant cache leakage?

01 SHORT INTERVIEW ANSWER

Include tenant and all authorization-relevant dimensions in identity, and authorize the source/read path correctly.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
product-summary:v1:tenant-7:user-scope:42
```

04 HOW IT WORKS, IN PLAIN ENGLISH

A cached value must not bypass permissions. If access depends on changing roles or ownership, choose invalidation, short lifetime or fresh authorization checks. Avoid caching one user's private response under an entity-only public key.

05 COMMON INTERVIEW TRAP

Adding tenantId alone does not cover user-specific permissions.

06 FOLLOW-UP: WHAT HAPPENS WHEN ACCESS IS REVOKED?

Recheck current authorization at the correct boundary and invalidate/version user-specific cached responses after revocation. A previously cached response must not grant rights that the live policy denies.

SENIOR ARCHITECTURE AND DESIGN CHALLENGES

How do you roll out a new cache serialization format?

01 SHORT INTERVIEW ANSWER

Use a compatible reader or versioned namespace, then deploy and warm gradually with a rollback plan.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
v1 readers + writers -> coexist
   v2 namespace -> gradual warm -> retire v1 after safe window
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Versioned keys reduce decoding conflicts but temporarily increase memory and source load. Do not delete old state before the rollback window if it is required. Session serialization changes can log users out or break reads and need stricter planning.

05 COMMON INTERVIEW TRAP

A rolling deployment means old and new code share data concurrently.

06 FOLLOW-UP: WHAT WOULD YOU MONITOR DURING THE ROLLOUT?

Monitor deserialization errors, old/new namespace hit rates, data size, latency, fallback load and rollback-read compatibility. Keep v1 readers working until all mixed-version instances are retired.

SENIOR ARCHITECTURE AND DESIGN CHALLENGES

What makes an architect-level Redis interview answer strong?

01 SHORT INTERVIEW ANSWER

State the business invariant, selected structure/protocol, failure ambiguity, recovery policy and verification evidence.

02 VISUAL MENTAL MODEL



03 ORIGINAL PRACTICAL EXAMPLE

```
Invariant: one payment effect per paymentId
   Redis: optional acceleration; DB/provider enforce identity
   Evidence: timeout, duplicate, failover and concurrent-request tests
```

04 HOW IT WORKS, IN PLAIN ENGLISH

Discuss memory, TTL, topology, consistency and ownership. Distinguish cache availability from critical-state safety. Use measured load and explain the limits of locks or asynchronous replicas rather than claiming universal exactly-once behavior.

05 COMMON INTERVIEW TRAP

Listing Redis commands without a failure model is not a design.

06 FOLLOW-UP: WHICH ASSUMPTION WOULD YOU CHALLENGE FIRST?

Challenge the claimed guarantee first: Is Redis the source of truth or only an accelerator? Then ask what must remain correct after timeouts, failover, stale cache writes and duplicate requests; propose tests that prove it.

CODING WORKSHOP

10 source-based Redis labs

From atomic expiry and cache key design to leases, Lua, Cluster and reactive cache-aside.

IMPORTANT VERIFICATION BOUNDARY

These are the original guide's worked snippets. Several require application-specific types and dependencies; they are not claimed to be standalone compilable projects. The companion ZIP includes a reconstructed first demo plus the ten original lab extracts.

Practice targets

C01
Atomic value and expiry

C02
An explicit Redis cache manager

C03
Verify the proxy and tenant-safe cache key

C04
Commit-aware invalidation and its remaining race

C05
Atomic fixed-window limiter

C06
Basic lease acquisition and ownership-safe release

C07
Cluster-compatible atomic transfer sketch

C08
Bounded pipelining with clear semantics

C09
Reactive cache-aside with an explicit failure policy

C10
Integration and failure-test exercise

C01 / PRACTICAL EXERCISE

Atomic value and expiry

GOAL AND SETUP

Store a string with a deadline in one operation. Setup: StringRedisTemplate; Boot Redis starter

ORIGINAL WORKED SOLUTION

```
public void store(String id, String json) {
  strings.opsForValue().set("demo:product:v1:" + id,
      json, Duration.ofSeconds(60));
}
public String read(String id) {
  return strings.opsForValue().get("demo:product:v1:" + id);
}
```

VERIFICATION AND LIMITS

Inject StringRedisTemplate through a constructor. Validate tenant/key inputs for the real application. Verify the key bytes, value and positive PTTL. Test overwrite with and without a Duration. A timeout may mean the SET already happened. A separate SET and EXPIRE would have a crash gap this operation avoids.

C02 / PRACTICAL EXERCISE

An explicit Redis cache manager

GOAL AND SETUP

Configure named caches with string keys and a fixed JSON DTO format. Setup: Boot Redis/ cache starters; Jackson; @EnableCaching

ORIGINAL WORKED SOLUTION

```
@Bean
RedisCacheManager cacheManager(RedisConnectionFactory factory) {
  var base = RedisCacheConfiguration.defaultCacheConfig()
      .entryTtl(Duration.ofMinutes(5))
      .disableCachingNullValues()
      .serializeKeysWith(RedisSerializationContext.SerializationPair
          .fromSerializer(new StringRedisSerializer()))
      .serializeValuesWith(RedisSerializationContext.SerializationPair
          .fromSerializer(new Jackson2JsonRedisSerializer<>(Product.class)));
  var writer = RedisCacheWriter.nonLockingRedisCacheWriter(
      factory, BatchStrategies.scan(1000));
  return RedisCacheManager.builder(writer)
      .cacheDefaults(base)
      .withInitialCacheConfigurations(Map.of(
          "products", base,
          "shortProducts", base.entryTtl(Duration.ofSeconds(30))))
      .transactionAware().build();
}
```

VERIFICATION AND LIMITS

The SCAN batch strategy is supported with Lettuce in the baseline; review driver/topology support before changing it. Product is a simple DTO supplied by the application; use this manager only for caches storing that DTO. Provide suitable Jackson modules/ObjectMapper if the DTO needs them. In the 3.5 baseline the default cache key prefix remains products::, separate from template keys. transactionAware defers supported cache changes around successful Spring-managed transactions; it does not create a SQL/Redis atomic commit. Verify JSON bytes, TTL, cache names, rollback behavior and stored old-schema values. Do not reuse this fixed Product serializer for arbitrary session or counter objects.

C03 / PRACTICAL EXERCISE

Verify the proxy and tenant-safe cache key

GOAL AND SETUP

Cache a read with all required tenant identity dimensions. Setup: @EnableCaching; RedisCacheManager; Spring-managed service

ORIGINAL WORKED SOLUTION

```
@Cacheable(cacheNames="products",
    key="#p0 + ':' + #p1")
public Product find(String tenantId, String productId) {
  return repository.loadAuthorized(tenantId, productId);
}
// External caller receives this service through Spring injection.
// Same tenant/id -> hit; different tenant -> separate key.
```

VERIFICATION AND LIMITS

Use parameter positions so the sample does not depend on parameter-name metadata. Define canonical input identifiers that cannot create ambiguous composite keys; use a custom KeyGenerator if necessary. Authorize each access according to actual policy: caching a response must not bypass fresh permission checks that are required on hits. Test load invocation count, two tenants and self-invocation bypass. This method assumes the chosen tenant-level representation is shareable within the authorized scope.

C04 / PRACTICAL EXERCISE

Commit-aware invalidation and its remaining race

GOAL AND SETUP

Evict after a successful data update and reproduce a delayed stale loader. Setup: SQL transaction; transaction-aware cache manager; separate proxied services

ORIGINAL WORKED SOLUTION

```
@Transactional
@CacheEvict(cacheNames="products", key="#p0 + ':' + #p1")
public void update(String tenantId, String id, ProductChange change) {
  repository.update(tenantId, id, change);
}
// Configure transaction-aware cache behavior and verify advice path.
// Race to reproduce:
// reader loads old DB data -> writer commits/evicts -> reader caches old
```

VERIFICATION AND LIMITS

ProductChange and repository are application code. Use integration tests for successful commit and rollback, then use barriers to force the stale-loader race. Transaction-aware eviction improves commit alignment but is not proof of strict consistency. Choose a version-aware protocol or authoritative reads for strict requirements; otherwise document bounded staleness and repair. Do not add a second deletion with an arbitrary sleep and declare all races solved.

C05 / PRACTICAL EXERCISE

Atomic fixed-window limiter

GOAL AND SETUP

Create/increment a per-window counter with an expiry in a short script. Setup: StringRedisTemplate; Lua; key includes trusted identity and fixed time bucket

ORIGINAL WORKED SOLUTION

```
String lua = "local n=redis.call('INCR',KEYS[1]); "
    + "if n==1 then redis.call('PEXPIRE',KEYS[1],ARGV[1]); end; "
    + "return n";
RedisScript<Long> script = new DefaultRedisScript<>(lua, Long.class);
Long count = strings.execute(script,
    List.of("limit:user-42:minute-123"), "120000");
boolean allowed = count != null && count <= 100;
```

VERIFICATION AND LIMITS

The application computes a trusted shared time-bucket identity and supplies a validated positive TTL longer than the bucket plus allowed clock/network margin. This script is fixed-window counting, not exact sliding-window admission. Rejected requests also increment this counter. Test concurrency, bucket-boundary bursts, bad types, outage policy and unknown response outcomes. In Cluster it uses one key, so it does not require cross-slot mutation.

C06 / PRACTICAL EXERCISE

Basic lease acquisition and ownership-safe release

GOAL AND SETUP

Use conditional SET with TTL and compare-and-delete; state the safety limits. Setup: StringRedisTemplate; unique random token per acquisition

ORIGINAL WORKED SOLUTION

```
String key = "lease:job:42";
String token = UUID.randomUUID().toString();
boolean acquired = Boolean.TRUE.equals(strings.opsForValue()
    .setIfAbsent(key, token, Duration.ofSeconds(10)));
String releaseLua = "if redis.call('GET',KEYS[1]) == ARGV[1] then "
    + "return redis.call('DEL',KEYS[1]) else return 0 end";
RedisScript<Long> release = new DefaultRedisScript<>(releaseLua, Long.class);
// Run only for the token owned by this attempt:
strings.execute(release, List.of(key), token);
```

VERIFICATION AND LIMITS

The exercise demonstrates a basic single-store lease, not universal mutual exclusion. Execute business work only after successful acquisition and handle release in the proper lifecycle. Test A expiry, B acquisition and late A release: B's key must survive. Then demonstrate how A could still perform an external write after its lease expired. Protect the resource with a real invariant/ fencing design where required. Failover, pauses and unknown acquisition outcomes remain explicit design concerns.

C07 / PRACTICAL EXERCISE

Cluster-compatible atomic transfer sketch

GOAL AND SETUP

Colocate required keys and validate before mutating them. Setup: StringRedisTemplate; integer values; two keys in one hash slot

ORIGINAL WORKED SOLUTION

```
String lua = "local a=tonumber(redis.call('GET',KEYS[1])); "
    + "local b=tonumber(redis.call('GET',KEYS[2])); "
    + "local n=tonumber(ARGV[1]); "
    + "if not a or not b or not n or n<=0 then return -1 end; "
    + "if a<n then return 0 end; "
    + "redis.call('SET',KEYS[1],a-n); "
    + "redis.call('SET',KEYS[2],b+n); return 1";
var script = new DefaultRedisScript<Long>(lua, Long.class);
Long result = strings.execute(script,
    List.of("wallet:{tenant7}:a", "wallet:{tenant7}:b"), "10");
```

VERIFICATION AND LIMITS

This is a small educational integer-state exercise, not a financial ledger. Restrict input ranges and validate exact integer representation; Lua numeric precision is unsuitable for arbitrary large balances. SET changes TTL unless explicitly handled. Test missing/invalid values, insufficient funds, same key rejection at the application boundary and CROSSSLOT without the tag. Concurrent script execution is serialized, but runtime errors do not generally roll back prior mutations, and failover durability is separate. Production money transfers need an authoritative durable transaction design and stable operation identity.

C08 / PRACTICAL EXERCISE

Bounded pipelining with clear semantics

GOAL AND SETUP

Send independent reads together without claiming atomicity. Setup: StringRedisTemplate; Spring Redis callback API

ORIGINAL WORKED SOLUTION

```
List<Object> results = strings.executePipelined(
    (RedisCallback<Object>) connection -> {
      connection.stringCommands().get(
          "product:42".getBytes(StandardCharsets.UTF_8));
      connection.stringCommands().get(
          "product:43".getBytes(StandardCharsets.UTF_8));
      return null;
    });
```

VERIFICATION AND LIMITS

StringRedisTemplate applies its configured result deserialization. The callback returns null because pipeline results are collected separately. Test ordering of replies, a missing key and mixed result/error behavior for the selected driver. Benchmark a bounded batch against separate calls. Other clients can mutate keys between these reads; this is not a consistent snapshot. Cluster routing and supported commands must be tested separately before applying the pattern there.

C09 / PRACTICAL EXERCISE

Reactive cache-aside with an explicit failure policy

GOAL AND SETUP

Compose a lazy source read and propagate Redis errors unless a reviewed fallback is added. Setup: ReactiveStringRedisTemplate; nonblocking source; Reactor

ORIGINAL WORKED SOLUTION

```
Mono<String> find(String key) {
  return redis.opsForValue().get(key)
      .switchIfEmpty(Mono.defer(() -> source.loadJson(key)
          .flatMap(value -> redis.opsForValue()
              .set(key, value, Duration.ofSeconds(30))
              .flatMap(saved -> saved
                  ? Mono.just(value)
                  : Mono.error(new IllegalStateException("Not cached"))))));
}
```

VERIFICATION AND LIMITS

This exercise deliberately propagates Redis get/write failures. source.loadJson returns Mono<String> and remains nonblocking. A source-backed endpoint may choose to return valid source data after selected cache-write failures, but implement and test that policy explicitly. Test hit, miss, empty source, write failure, cancellation and simultaneous cold misses. The composition does not provide cross-pod single-flight. Do not call block on the event-loop path.

C10 / PRACTICAL EXERCISE

Integration and failure-test exercise

GOAL AND SETUP

Prove key lifecycle, cached invocation and failure behavior with a real Redis fixture. Setup: Testcontainers or isolated Redis; actual Spring proxy; bounded await

ORIGINAL WORKED SOLUTION

```
// Fixture supplies strings and proxied ProductService.
service.find("tenant-7", "42");
service.find("tenant-7", "42");
assertThat(repositoryLoadCount()).isEqualTo(1);
assertThat(strings.getExpire("products::tenant-7:42", TimeUnit.MILLISECONDS))
    .isPositive();
await().atMost(Duration.ofSeconds(10)).untilAsserted(() ->
    assertThat(strings.hasKey(testExpiringKey())).isFalse());
```

VERIFICATION AND LIMITS

Set a short TTL for the tested cache/key and use unique fixture identities. Repository count can be an instrumented real adapter or a controlled loader, while serialization and cache storage use actual Redis. Awaitility/assertion imports and fixture helpers are required; this is a test sketch. Add tests for cache-write failure, rollback, duplicate limiter requests, late lease release, stale-loader race and serializer migration. A one-node fixture cannot prove Cluster or failover behavior: use appropriate additional infrastructure for those tests.

FAST REVISION

Redis commands at a glance

REMEMBER THE GUARANTEE, NOT JUST THE SYNTAX

Fast execution or one atomic Redis command does not automatically make the downstream database effect durable.

Command / Concept	Meaning	Interview warning
SET key value EX 60	Write with TTL	Unknown response can still mean success
TTL / PTTL	Remaining lifetime	-1 no expiry; -2 missing
INCR	Atomic counter	Retries can double-increment
SET NX PX	Lease attempt	Expiry does not fence stale holder
MULTI / EXEC	Queued command transaction	Not SQL-style rollback on runtime error
WATCH	Optimistic conflict watch	Must retry after conflict
SCAN	Cursor key iteration	Can repeat keys during changes
GETEX	Get and modify expiry	Not same as ordinary GET
XPENDING / XAUTOCLAIM	Stream recovery	Pending entry needs durable-effect handling
MOVED / ASK	Cluster routing	All advertised endpoints must be reachable

PRODUCTION CHECKLIST

The architect's Redis checklist

Source of truth

If Redis disappears, can the app still reconstruct the data safely?

Freshness

What stale-data budget is acceptable? Can a late writer extend it?

Identity

Do keys include tenant, permission scope and schema version?

Failure mode

What happens after a timeout, failover, GC pause or duplicate request?

Capacity

Are hot keys, memory, connection pools and fallback loads bounded?

Proof

Which integration tests show the real command, TTL, serialization and failover behavior?

Speak this in interviews: Define the requirement → show the operation → identify the race/crash window → choose a repair → name the test proving it.

30-DAY STUDY ROUTE

Practice days 1–10

One goal per day, with an explanation you can give without notes.

Day	Focus	Required outcome
01	Redis purpose	Separate cache data from authoritative state
02	Data structures	Choose strings, hashes, sets and sorted sets
03	Keys and TTL	Complete C01; explain -1/-2 TTL
04	Boot setup	Run first application with Redis
05	Serialization	Inspect bytes and design a versioned DTO namespace
06	Cache annotations	Complete C03; show proxy bypass
07	Checkpoint	Explain Q001-Q030 without notes
08	Cache manager	Complete C02; inspect different TTLs
09	TTL and TTI	Contrast GET and GETEX
10	Invalidation	Complete C04 and reproduce stale-loader race

Suggested daily session: 40 minutes concepts, 40 minutes examples and code, 20 minutes speaking answers, 20 minutes reviewing mistakes.

30-DAY STUDY ROUTE

Practice days 11–20

One goal per day, with an explanation you can give without notes.

Day	Focus	Required outcome
11	Miss overload	Bound fallback and jitter
12	Pipelining	Complete C08; show interleaving
13	Transactions and scripts	Explain MULTI and Lua failure limits
14	Checkpoint	Answer Q031-Q050
15	Basic leases	Complete C06; test late release
16	Fencing	Protect downstream resource from stale writer
17	Rate limiting	Complete C05; simulate boundary burst
18	Business idempotency	Draw marker/effect crash gaps
19	Redis Cluster	Complete C07; hash-tag test
20	Routing	Diagnose MOVED and ASK

Suggested daily session: 40 minutes concepts, 40 minutes examples and code, 20 minutes speaking answers, 20 minutes reviewing mistakes.

30-DAY STUDY ROUTE

Practice days 21–30

One goal per day, with an explanation you can give without notes.

Day	Focus	Required outcome
21	Checkpoint	Answer Q051-Q070 including failover
22	Memory and latency	Review eviction and big keys
23	Sessions and messaging	Compare Pub/Sub, Streams and Kafka
24	Reactive access	Complete C09; errors vs empty results
25	Testing	Complete C10 with real Redis
26	Security and health	Define ACL/TLS/outage policies
27	Senior design	Two-level caching and tenant-safe identity
28	Mock interview A	Implementation and failures, 45 minutes
29	Mock interview B	Catalog + sessions + limiter, 60 minutes
30	Final review	Repeat weak answers; show test evidence

Suggested daily session: 40 minutes concepts, 40 minutes examples and code, 20 minutes speaking answers, 20 minutes reviewing mistakes.

FINAL REHEARSAL

Two mock interview formats

Mock A — implementation (45 minutes)

Explain cache configuration, one cache-miss race, a safe lease release, a Redis Cluster slot issue and a real TTL test. Write code or commands for two of them.

Mock B — senior architecture (60 minutes)

Design a product cache, browser session store and rate limiter. Specify trusted source, staleness limit, failover plan, security boundaries and negative tests for each.

Release-ready reasoning

For every critical promise, produce a reproducible failure test—not just a successful demo. Explain what Redis guarantees and what the database or business protocol must enforce.